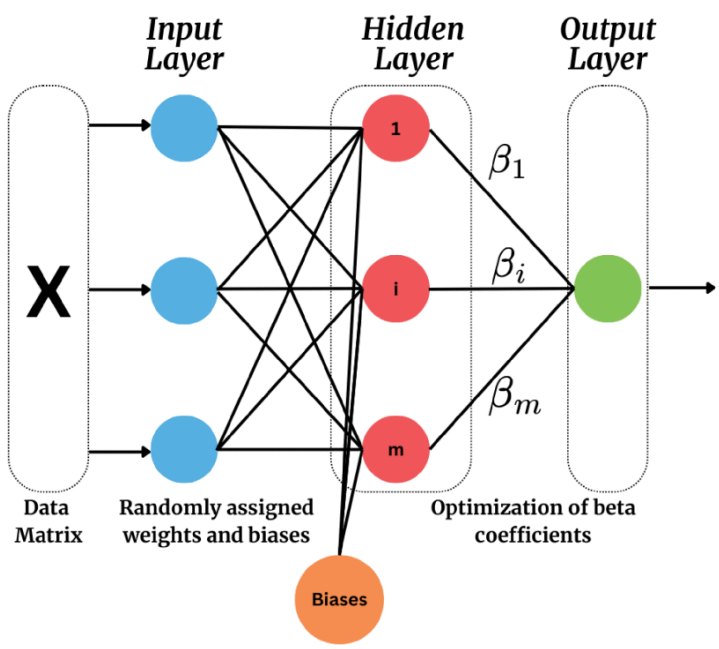
# Project Report: Implementing Regularised Extreme Learning Machines for Multicollinear Data

Allen Sumanth A (108123012), Adithya S(108123006)
Github link

## Introduction

Machine learning models struggle when input features are highly correlated. This issue is called multicollinearity. It causes standard algorithms to become unstable and produce wildly inaccurate predictions. This project replicates a recent study proposing two novel algorithms designed to fix this fundamental flaw. The project is based on the research paper "Novel Statistical Regularized Extreme Learning Algorithm to Address the Multicollinearity in Machine Learning" by H. Yildirim (IEEE Access, vol. 12, pp. 102355-102367, 2024, doi: 10.1109/ACCESS.2024.3432490). We built a testing framework similar to the one implemented (with a few negligible compromises) in the paper from scratch to evaluate these new methods against existing industry standards across multiple datasets.

## The Basics of Extreme Learning Machines




*The general representation of the basic ELM model.*

The Universal Approximation Theorem states that a single layer feed forward network can approximate any mathematical function. Researchers have proved that a network requires exactly N hidden nodes to perfectly match N data points. Traditional neural networks find the right connection weights by slowly

tweaking them over many training cycles.
Extreme Learning Machines take a completely different approach. They randomly assign weights to the hidden layer and lock them in place. The network then calculates the final output weights in a single mathematical step using linear algebra.
The core relationship is defined by the equation $H\beta = Y$. Here $H$ represents the hidden layer output matrix. The symbol $\beta$ represents the output weight vector we want to find. The letter $Y$ represents the target values. To solve for $\beta$, the system must isolate it. Because $H$ is rarely a perfect square matrix, standard inversion is impossible. The baseline model solves this by computing the Moore-Penrose pseudoinverse denoted as $H^{\dagger}$. The final calculation becomes $\hat{\beta}_{ELM} = H^{\dagger}Y$. This design makes the training process incredibly fast compared to iterative gradient descent.

# The Issue of Multicollinearity

The raw speed of standard Extreme Learning Machines comes with a major weakness. When the input data contains highly correlated features, the columns of the $H$ matrix become linearly dependent. This dependence causes the $H^T H$ matrix to become nearly singular. Its determinant approaches zero.
Inverting a nearly singular matrix breaks the math completely. The resulting output weights explode in size. A model with massive weights will overreact to tiny fluctuations in new data causing extreme prediction errors.

# Single Parameter Regularisation

Researchers previously introduced single parameter penalties to stabilize the math. The Ridge estimator solves the singular matrix problem by adding a positive penalty value $k$ directly to the matrix diagonal.
$\hat{\beta}_{Ridge-ELM} = (H^T H + kI_m)^{-1} H^T Y$
Adding this $k$ value guarantees the matrix remains invertible. It physically shrinks the overall size of the resulting weights.
The Liu estimator takes a slightly different path. It applies a shrinkage penalty $d$ to control the variance of the estimates.
$\hat{\beta}_{Liu-ELM} = (H^T H + I_m)^{-1}(H^T Y + d\hat{\beta}_{ELM})$

# The Proposed Two-Parameter Algorithms

Relying on a single parameter forces a compromise between reducing bias and controlling variance. The paper we replicated introduces two new models named TP1 ELM and TP2 ELM to solve this exact limitation. These estimators simultaneously apply both the $k$ and $d$ penalties in new mathematical arrangements.
The first novel algorithm builds directly on the Ridge estimator by incorporating the $d$ parameter.
$\hat{\beta}_{TP1-ELM}(k,d) = (H^T H + kI_m)^{-1}(H^T Y + d\hat{\beta}_{Ridge-ELM})$
The second algorithm introduces a more complex multiplication chain to tightly control the output weights.
$\hat{\beta}_{TP2-ELM}(k,d) = (H^T H + I_m)^{-1}(H^T H + dI_m)(H^T H + kI_m)^{-1} H^T Y$

This dual parameter approach forces the matrix to remain stable while keeping the prediction errors as low as possible.

# Project Engineering and Implementation Details

We built the entire testing environment in Python. We applied specific engineering choices to ensure the code ran quickly and correctly on a standard local processor.

## CPU Optimisation Strategy

The baseline ELM requires a complex mathematical operation called a pseudoinverse to handle unstable matrices. Calculating a pseudoinverse is extremely slow. The regularised models add a positive penalty directly to the matrix diagonal. This penalty mathematically guarantees the matrix is invertible. We swapped the slow pseudoinverse for a fast direct algebraic solver that uses this property to its advantage. This choice allowed the entire experiment to run on a standard processor in minutes instead of relying on expensive cloud graphics cards.

## Preventing Data Leakage

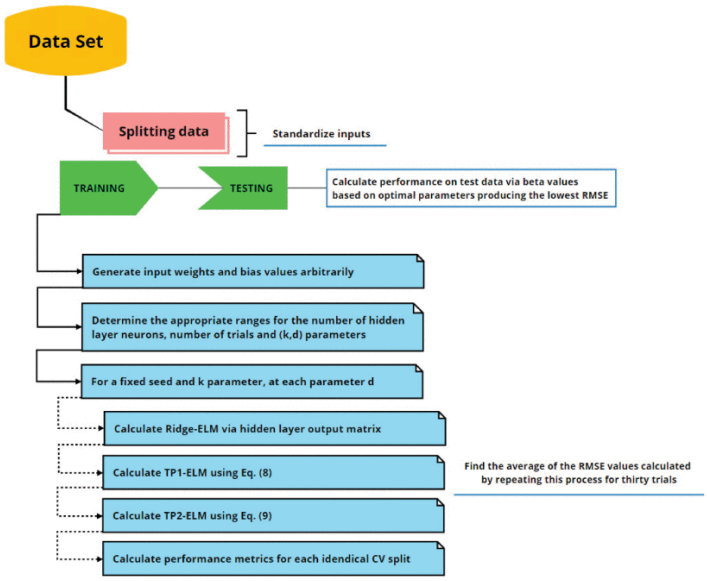
A common mistake in machine learning is tuning a model using the final test data. We built an inner validation split within the training loop to prevent this error. The algorithms tested different penalty combinations on this isolated validation data to find the best settings. The final models were then evaluated on completely unseen test data to ensure the performance metrics remained honest.

## Practical Scope Adjustments

The original research paper included a massive array of tests. We scaled the experiment down to fit a realistic timeframe while preserving the core scientific integrity. We reduced the number of traditional comparison models from seventeen to three. We compared the new algorithms against Ridge Regression, Random Forest, and XGBoost. We also replaced the exhaustive parameter search with a randomised search testing fifteen parameter combinations. The project used a standard five-fold cross-validation loop rather than a 5-times repeated five fold loop.

# Experimental Setup

We ran the models across seven standard regression datasets. We manually fetched the Auto Price and Slump datasets from their original sources to fix broken download links in the standard library. We calculated the Variance Inflation Factor for each dataset. This mathematical check proved that severe multicollinearity existed in almost all the tested data. We ran thirty independent trials for every model to account for the random nature of the initial network weights.




*Flow chart describing the process*

# Results and Analysis

The test results strongly aligned with the original research findings.

**Predictive Accuracy**

Both the TP1 and TP2 models consistently produced lower error rates than the standard baseline ELM. The percentage improvements against the baseline model across four key datasets are clear:

| **Dataset** | **TP1 Improvement (%)** | **TP2 Improvement (%)** |
|---|---|---|
| Machine CPU | 99.75 | 99.76 |
| Auto Price | 54.46 | 58.61 |
| Servo | 39.99 | 41.57 |
| Slump | 20.12 | 30.75 |

Traditional tree algorithms like Random Forest and XGBoost still produced lower raw error scores overall. However, the new ELM models succeeded in their primary goal of fixing the broken matrix math and closing the massive performance gap. The new models remained highly competitive with heavy traditional algorithms while requiring a fraction of the training time.

**Predictive Accuracy Comparison (Reduction Rate)Comprehensive Model Performance Metrics**

Reduction Rate (RR%)

| Dataset | Proposed | BaseELM | RidgeELM | LiuELM | OK_ELM | Ridge | RandomForest | XGBoost |
|---|---|---|---|---|---|---|---|---|
| Auto Price | TP1_ELM | 54.46% | -0.37% | -1.58% | 21.35% | -315.80% | -446.74% | -449.27% |
| | TP2_ELM | 58.61% | 8.76% | 7.67% | 28.51% | -277.96% | -396.98% | -399.28% |
| Boston Housing | TP1_ELM | 1.16% | -0.02% | -0.02% | -0.01% | -210.19% | -346.77% | -376.84% |
| | TP2_ELM | 2.16% | 0.99% | 0.99% | 1.00% | -207.06% | -342.25% | -372.02% |
| Fish | TP1_ELM | 3.68% | -0.22% | -0.28% | -0.25% | -28.64% | -36.04% | -34.87% |
| | TP2_ELM | 4.53% | 0.66% | 0.60% | 0.63% | -27.51% | -34.84% | -33.68% |
| Forests | TP1_ELM | 0.37% | -0.01% | -0.01% | -0.01% | -22.31% | -13.19% | -12.73% |
| | TP2_ELM | 0.75% | 0.37% | 0.37% | 0.37% | -21.84% | -12.76% | -12.30% |
| Machine CPU | TP1_ELM | 99.75% | -1.34% | 36.51% | 56.58% | -25.05% | -17.13% | -18.56% |
| | TP2_ELM | 99.76% | 4.57% | 40.21% | 59.11% | -17.76% | -10.30% | -11.65% |
| Servo | TP1_ELM | 39.99% | -0.09% | -0.27% | 0.18% | -24.53% | -68.25% | -94.88% |
| | TP2_ELM | 41.57% | 2.55% | 2.37% | 2.81% | -21.25% | -63.82% | -89.75% |
| Slump | TP1_ELM | 20.12% | -12.64% | -13.39% | -12.70% | -551.15% | -365.26% | -408.36% |
| | TP2_ELM | 30.75% | 2.35% | 1.70% | 2.30% | -464.49% | -303.35% | -340.71% |
| Strikes | TP1_ELM | 7.05% | -0.51% | -0.49% | -0.44% | -12.13% | -8.92% | -10.17% |
| | TP2_ELM | 9.93% | 2.60% | 2.62% | 2.67% | -8.66% | -5.55% | -6.76% |

The Reduction Rate measures the percentage change in the prediction error between two models. The formula subtracts the proposed model error from the baseline model error. It then divides that result by the baseline model error and multiplies by one hundred. A positive percentage means the proposed model successfully reduced the error and performed better than the baseline. A negative percentage means the proposed model generated a higher error than the comparison model.

**Comprehensive Model Performance Metrics**

Full Results (Norm = Mean ± Std)

| | BaseELM \| Train RMSE | BaseELM \| Test RMSE | BaseELM \| Norm | RidgeELM \| Train RMSE | RidgeELM \| Test RMSE | RidgeELM \| Norm | LiuELM \| Train RMSE | LiuELM \| Test RMSE | LiuELM \| Norm | OK_ELM \| Train RMSE | ... | TP2_ELM \| Norm | Ridge \| Train RMSE | Ridge \| Test RMSE | Ridge \| Norm | RandomForest \| Train RMSE | RandomForest \| Test RMSE | RandomForest \| Norm | XGBoost \| Train RMSE | XGBoost \| Test RMSE | XGBoost \| Norm |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Auto Price** | 1053.2533 ± 144.2096 | 25011.0056 ± 4697.3369 | 64428.7174 ± 8464.9479 | 1915.0152 ± 222.5482 | 11346.9151 ± 1253.0378 | 24372.2387 ± 2252.7006 | 1947.0071 ± 195.6867 | 11212.6067 ± 1225.2009 | 23752.3990 ± 1247.7040 | 1555.0300 ± 240.7876 | ... | 20670.7861 ± 1495.5639 | 2207.5822 ± 18.0156 | 2739.1416 ± 103.7826 | 3554.4250 ± 316.2556 | 1010.1970 ± 124.3703 | 2083.1541 ± 104.6628 | N/A | 485.4099 ± 318.2224 | 2073.5269 ± 173.5094 | N/A |
| **Boston Housing** | 9.6920 ± 0.7856 | 15.1660 ± 1.3113 | 39.0117 ± 2.4572 | 9.6986 ± 0.7855 | 14.9872 ± 1.2874 | 37.6511 ± 2.2323 | 9.6986 ± 0.7855 | 14.9872 ± 1.2874 | 37.6512 ± 2.2323 | 9.6984 ± 0.7855 | ... | 36.4356 ± 2.0425 | 4.5678 ± 0.0082 | 4.8325 ± 0.0495 | 7.4756 ± 0.3349 | 1.5677 ± 0.1337 | 3.3552 ± 0.1540 | N/A | 0.7345 ± 0.5124 | 3.1436 ± 0.2059 | N/A |
| **Fish** | 0.8334 ± 0.0403 | 1.2749 ± 0.1343 | 9.8780 ± 1.4495 | 0.8430 ± 0.0408 | 1.2253 ± 0.1205 | 7.7818 ± 0.9900 | 0.8433 ± 0.0407 | 1.2246 ± 0.1221 | 7.7675 ± 0.9855 | 0.8428 ± 0.0408 | ... | 7.0728 ± 0.9693 | 0.9447 ± 0.0003 | 0.9546 ± 0.0032 | 0.9137 ± 0.0031 | 0.4880 ± 0.0442 | 0.9027 ± 0.0104 | N/A | 0.5130 ± 0.1441 | 0.9105 ± 0.0142 | N/A |
| **Forests** | 55.9718 ± 0.9332 | 66.7399 ± 1.8228 | 55.5879 ± 2.7999 | 55.9727 ± 0.9332 | 66.4840 ± 1.8223 | 54.8308 ± 2.7524 | 55.9727 ± 0.9332 | 66.4840 ± 1.8223 | 54.8309 ± 2.7524 | 55.9727 ± 0.9332 | ... | 54.0992 ± 2.7083 | 60.8944 ± 0.6396 | 54.3633 ± 2.2408 | 38.1715 ± 10.4292 | 38.5426 ± 2.1850 | 58.7417 ± 2.9090 | N/A | 39.1354 ± 13.5247 | 58.9805 ± 8.6072 | N/A |
| **Machine CPU** | 3.1638 ± 0.0985 | 7998.9297 ± 5234.6181 | 5509903.3340 ± 1995669.4848 | 13.2663 ± 0.2605 | 19.9509 ± 1.7193 | 53.4335 ± 42.5382 | 13.3858 ± 0.1103 | 31.8446 ± 43.8325 | 6662.0237 ± 7798.2571 | 12.2892 ± 0.3176 | ... | 35.7833 ± 14.9377 | 15.5345 ± 0.0344 | 16.1675 ± 0.3114 | 3.8432 ± 0.3560 | 12.1753 ± 0.6451 | 17.2609 ± 0.5681 | N/A | 11.6916 ± 2.6471 | 17.0530 ± 1.1756 | N/A |
| **Servo** | 0.3568 ± 0.0463 | 2.3102 ± 0.5452 | 14.3979 ± 1.7777 | 0.5147 ± 0.0616 | 1.3852 ± 0.1656 | 4.6594 ± 0.5187 | 0.5150 ± 0.0537 | 1.3826 ± 0.1762 | 4.6349 ± 0.3693 | 0.5099 ± 0.0574 | ... | 3.9692 ± 0.6246 | 1.0248 ± 0.0026 | 1.1133 ± 0.0169 | 2.0061 ± 0.0581 | 0.5700 ± 0.0485 | 0.8240 ± 0.0512 | N/A | 0.2862 ± 0.1306 | 0.7114 ± 0.0895 | N/A |
| **Slump** | 0.0000 ± 0.0000 | 21.3950 ± 5.5802 | 66.3250 ± 11.1209 | 1.9436 ± 0.5597 | 15.1723 ± 2.9944 | 39.9735 ± 5.2716 | 2.0247 ± 0.5705 | 15.0721 ± 3.0081 | 39.5969 ± 5.1897 | 1.9185 ± 0.5519 | ... | 36.3342 ± 4.4940 | 2.3330 ± 0.0436 | 2.6246 ± 0.0864 | 9.3620 ± 0.4037 | 1.5505 ± 0.1296 | 3.6732 ± 0.1534 | N/A | 0.3097 ± 0.3787 | 3.3618 ± 0.2763 | N/A |
| **Strikes** | 432.9100 ± 5.6379 | 612.5706 ± 21.2774 | 3634.4354 ± 502.1274 | 438.4344 ± 5.4526 | 566.4720 ± 16.6945 | 1872.7070 ± 194.5223 | 438.4078 ± 5.4848 | 566.5731 ± 16.8101 | 1877.2757 ± 198.5402 | 438.2869 ± 5.4689 | ... | 1396.5119 ± 214.3760 | 524.0912 ± 1.1782 | 507.7516 ± 11.0515 | 199.6056 ± 1.1686 | 358.5270 ± 25.1955 | 522.7271 ± 13.5947 | N/A | 382.9539 ± 105.6924 | 516.8154 ± 16.8844 | N/A |

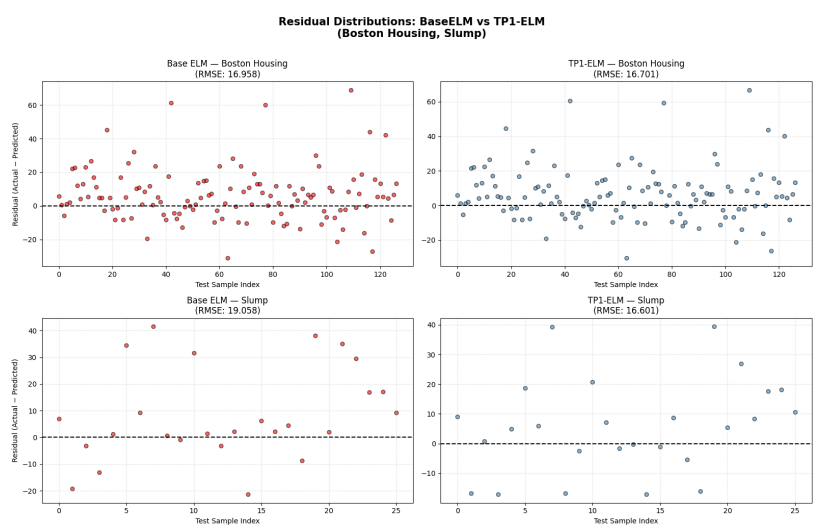
Ridge Regression, Random Forest, and XGBoost produced significantly lower raw error scores on several datasets. These strong traditional model performances resulted in the negative reduction rates seen for the regularised ELM models. While the new TP1 and TP2 models did not beat the heavy tree algorithms in absolute predictive accuracy, they successfully fixed the severe mathematical instability. This success is clear when observing the massive reduction in coefficient norms across all datasets, especially the Machine CPU and Auto Price tests.

### Weight Stability

The norm of the coefficient weights serves as a direct measure of model stability. The baseline ELM produced massive weight values across all thirty trials. For instance, the baseline ELM failed completely on the Machine CPU data, generating a coefficient norm of 5,509,903.33 and an error rate of 7,998.92. The baseline ELM also failed on the Auto Price data, generating a coefficient norm of 64,428.71 and an error rate of 25,011.00. The TP1 and TP2 models drastically shrank these weights. The standard deviation of these weights also dropped significantly. The new algorithms successfully stopped the math from breaking down under pressure.

### Residual Errors

We plotted the actual prediction errors for the Boston Housing and Slump datasets. The baseline ELM graphs show massive spikes where the model guessed entirely wrong values. The TP1 graphs show the errors clustered tightly around zero. The visual evidence proves the regularisation penalties function exactly as intended.




# Summary

This project successfully validated the claims of the source research. Building the regularised Extreme Learning Machines from scratch required careful attention to matrix algebra and strict validation practices. The final algorithms provide a fast and highly stable alternative to traditional models when

handling correlated data.

# Sources


1. H. Yildirim, "Novel Statistical Regularized Extreme Learning Algorithm to Address the Multicollinearity in Machine Learning," in IEEE Access, vol. 12, pp. 102355-102367, 2024, doi: 10.1109/ACCESS.2024.3432490. https://ieeexplore.ieee.org/document/10606466
2. Universal approximation theorem: https://en.wikipedia.org/wiki/Universal_approximation_theorem
3. Guang-Bin Huang and H. A. Babri. 1998. Upper bounds on the number of hidden neurons in feedforward networks with arbitrary bounded nonlinear activation functions. Trans. Neur. Netw. 9, 1 (January 1998), 224–229. https://doi.org/10.1109/72.655045
4. Extreme learning machines: Guang-Bin Huang, Qin-Yu Zhu and Chee-Kheong Siew, "Extreme learning machine: a new learning scheme of feedforward neural networks," 2004 IEEE International Joint Conference on Neural Networks (IEEE Cat. No.04CH37541), Budapest, Hungary, 2004, pp. 985-990 vol.2, doi: 10.1109/IJCNN.2004.1380068. https://ieeexplore.ieee.org/document/1380068
5. H. Yildirim and M. Revan Özkale, ‘‘The performance of ELM based ridge regression via the regularization parameters,’’ Expert Syst. Appl., vol. 134, pp. 225–233, Nov. 201 https://www.sciencedirect.com/science/article/pii/S0957417419303744?via%3Dihub


# Annexures

1. Original Research Paper: "Novel Statistical Regularized Extreme Learning Algorithm to Address the Multicollinearity in Machine Learning" by H. Yildirim (IEEE Access, vol. 12, pp. 102355-102367, 2024, doi: 10.1109/ACCESS.2024.3432490).
2. Jupyter Notebook PDF Document.

IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 July 2024, accepted 19 July 2024, date of publication 23 July 2024, date of current version 1 August 2024.

*Digital Object Identifier 10.1109/ACCESS.2024.3432490*

RESEARCH ARTICLE

# Novel Statistical Regularized Extreme Learning Algorithm to Address the Multicollinearity in Machine Learning

**HASAN YILDIRIM**
Kamil Özdağ Faculty of Science, Department of Mathematics, Karamanoğlu Mehmetbey University, 70100 Karaman, Türkiye
e-mail: hasanyildirim@kmu.edu.tr

**ABSTRACT** The multicollinearity problem is a common phenomenon in data-driven studies, significantly affecting the performance of machine learning algorithms during the process of extracting information from data. Despite its widespread use across various fields, the extreme learning machine (ELM) also suffers from multicollinearity issues. To address this challenge, the ridge and Liu estimators, drawn from statistics literature, have been integrated into ELM theory, resulting in a notable advancement. This study aims to further enhance the capabilities of ridge and Liu estimators within the ELM framework by introducing two innovative two-parameter algorithms (TP1-ELM and TP2-ELM) that simultaneously incorporate both estimators. The proposed algorithms undergo comprehensive benchmarking against ELM, ELM-based algorithms, and other commonly used machine learning techniques across seven diverse datasets. Benchmark results demonstrate that the proposed algorithms consistently outperform both ELM-focused approaches and traditional machine learning algorithms on most datasets, yielding more generalizable and stable results. These findings suggest that the proposed algorithms offer a promising alternative to traditional machine learning techniques for regression and classification tasks, particularly in scenarios where multicollinearity is a concern.

**INDEX TERMS** Extreme learning machine, Liu estimator, machine learning, multicollinearity, ridge estimator, Tikhonov regularization.

## I. INTRODUCTION

In parallel with the facilitation of the acquisition of data, the need to make sense of it and transform it into valuable insights has emerged. For this purpose, numerous algorithms have been developed in the field of artificial intelligence, especially in the branch of machine learning. Artificial neural network-oriented algorithms stand out among these algorithms and continue to be intensively developed in the academic field.

Artificial neural networks have been used in practically every field due to its capabilities such as i) capturing complex patterns, ii) creating non-linear models, iii) flexibility. In general, artificial neural networks are classified into feed-forward and feed-back neural networks, with feed-forward networks leading the way with its simplicity [1], [2]. Feed-forward networks, studied first in the literature, have a number of parameters to be determined, such as learning rate, momentum, period, stopping criteria, input weights and biases. Huang et al. [3], [4], proposed the extreme learning machine (ELM) algorithm as a single-layer feed-forward neural network capable of being used in both regression and classification tasks, essentially eliminating the process of determining such parameters and avoiding risks related to slowness, local optimum and over-fitting.

The ELM algorithm assigns input layer weights and hidden layer parameters (e.g. biases) randomly and provides a closed-form solution, thereby achieving i) faster trainability, ii) reasonable generalization performance, iii) universal approximation capability, iv) applicability in machine learning domains such as regression, classification and clustering, and v) relatively low human intervention [5]. Due to the consequence of such capabilities, it has been used in a wide range of fields serving varied objectives. The main

The associate editor coordinating the review of this manuscript and approving it for publication was Diego Bellan.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

prominent studies can be presented as telecommunication [6], [7], [8], [9], [10], environmental sciences [11], [12], [13], robotics [14], [15], [16], computer sciences [17], [18], [19], [20], [21], agriculture [22], [23], management [24], [25], medical sciences [26], [27], [28], economics [29], [30], psychology [31], chemistry [32], [33], [34], [35] in Table 1. Therefore, the ELM algorithm, as a part of the feed-forward neural networks under artificial neural networks (also machine learning field), still appears to be an important algorithm in the process of transition from data to knowledge in this information age.

### A. LITERATURE REVIEW OF RELATED WORKS

The ELM algorithm has attracted considerable attention and been actively studied in the statistics literature due to the following reasons: i) linear model form, ii) closed-form model structure, iii) relying on classical least squares for model parameter estimation, iv) possession of the previously mentioned capabilities and v) a wide range of applications. However, the basic ELM algorithm remains highly appealing, it has a number of shortcomings arising from structural risk minimization [36]. The shortcomings can be summarized as follows: i) The risk of over-fitting models, ii) over-dependence on hidden layer structure, iii) lack of sparsity capability, iv) non-stable performance on new data. The main reason for the over-fitting and lack of stability is that the columns of the hidden layer matrix in the ELM algorithm are linearly dependent (i.e. correlated). In the statistics literature, this problem, known as multicollinearity, leads to extreme variability in the model predictions and weakening of the predictions obtained with new data (overfitting model).

Deng et al. [37] proposed a regularized ELM algorithm based on weighted least squares to overcome the difficulties related to structural risk minimization. Feng et al. [38] introduced the error minimized extreme learning machine (EM-ELM) algorithm based on incremental adjustment of hidden layer weights. Yuan et al. [39] improved the convergence performance of the basic ELM model by deriving an optimal model by examining the rank of the hidden layer matrix under different rank conditions. Lu et al. [40] presented a comparative study of different solutions of the Moore-Penrose matrix calculus used in the basic ELM solution, which is directly related to the structure of the hidden layer matrix. These studies mostly emphasize either derivation of the hidden layer matrix or optimization of the computations based on it. However, there are alternative efficient solutions to the multicollinearity problem for linear models based on a slight adjustment to the structure of the hidden layer matrix that have been proposed in the statistical literature. The most important of these approaches are the ridge estimator proposed by Hoerl and Kennard [41] and the Liu estimator proposed by Liu [42].

The two estimators were initially proposed by Deng et al. [37] and later presented in a unified framework by Huang et al. [43], [44] in order to strengthen the generalization performance and obtain a more robust result in the ELM context. Huang et al. [44] comprehensively described the theoretical and practical properties of ELM models based on the ridge estimator, along with its use in both classification and regression models with real-life data. Miche et al. [45] introduced a novel algorithm called OP-ELM to eliminate correlated or redundant components by considering the ridge estimator (a.k.a Tikhonov or $l_2$ norm regularization). Later, Miche et al. [46] developed the TROP-ELM algorithm, which is a modification of OP-ELM and incorporates LARS and Tikhonov regularization approaches into the model simultaneously. Martínez et al. [47] applied the ridge, lasso and elastic net approaches to the ELM domain and benchmarked the performances comparatively. Li and Niu [48] introduced novel models by extending ridge and alternative ridge-based estimators (such as almost unbiased ridge) to the ELM domain. Fakhr et al. [49] provided a comparative study of the basic ELM algorithm and its variants based on $l_1$ and $l_2$ norms. Luo et al. [50] also provide a detailed overview of the theoretical and practical properties of $l_1$ and $l_2$ form-based ELM algorithms for classification and regression problems. He et al. [51] proposed a novel Elm algorithm based on the $l_{1/2}$ norm to determine the number of hidden layer nodes and prune redundant components. Shan et al. [52] introduced a novel algorithm integrating the interval Lasso (i.e. $l_2$ norm) approach in the ELM algorithm, in order to achieve a more compact model selection. Yıldırım and Özkale [5] proposed ELM algorithms based on ridge, almost unbiased ridge and generalized ridge estimators, and also presented parameter selection methods (including cross validation (CV), Akaike information (AIC) and Bayesian information (BIC) criteria) with real life data.

The first implementation of the Liu estimator in the ELM domain is the Liu-ELM algorithm proposed by Yıldırım and Özkale [53], which is based on an improved form of ridge-based ELM algorithms. In order to address the lack of sparsity in the Liu estimator, a novel ELM algorithm called as LL-ELM based on simultaneous implementation with the Lasso approach has been presented by Yıldırım and Özkale [54]. Li and Zhao [55] present performance benchmarks by incorporating the Liu estimator into the Tensor-Based Type-2 Elm algorithm. The Ridge and Liu estimators based models represent a highly competitive ELM algorithm for real-world problems due to their different levels of model improvements. Therefore, the two-parameter estimator proposed by Özkale and Kaciranlar [56] in the statistics literature has been applied to the ELM domain and the OK-ELM algorithm has been proposed by Yıldırım and Özkale [57]. The proposed GO-ELM with this approach yields a better performance compared to the basic ELM, standalone ridge-based ELM and Liu-based ELM algorithms.

### B. RESEARCH MOTIVATION AND ENTHUSIASM

While the ridge estimator has seen extensive use in data-driven studies, particularly in machine learning, the Liu estimator is gaining recognition. The proposed GO-ELM integrates the capabilities of both estimators, offering a

robust alternative for classification and regression tasks. Two-parameter estimators are a prominent and thoroughly researched topic in statistical theory, with various alternative models available. These include the k-d estimator by Sakallıoğlu and Kaçranlar [58] and the y-c estimator by Yang and Chang [59]. In this paper, we introduce the first novel implementations of these two new two-parameter estimators within the ELM framework, providing the following properties:

- The proposed algorithms combine classical least squares, ridge, and Liu approaches to address the limitations of the ELM algorithm effectively.
- We present a comparative analysis of the performance of the two new proposed algorithms with the highly competitive GO-ELM algorithm using real-life applications.
- Detailed comparisons with the most popular machine learning algorithms demonstrate the relative superiority of the proposed algorithms.

The structure of the study is organized as follows: Section II summarizes the basic ELM algorithm and its properties. Section III briefly presents the algorithm and related models. Mathematical details and properties of the proposed algorithms are described in Section IV. Section V discusses application details and results with real-life data within a comprehensive framework. The last section presents the main conclusions of the study.

## II. THE PRELIMINARY ELM AND RELATED ALGORITHMS

ELM algorithm has been proposed for SLFNs and is effectively used in regression ans classification tasks. Supposed that $\left(\mathbf{x}_i^T, t_i^T\right), i = 1, \ldots, N$ are randomly taken samples from input space and $H = (h_1, h_2, \ldots, h_m)$ is the function set of explanatory variables (i.e basis functions), then a regression function can be defined as follows:

$$\widehat{f}(\mathbf{x}_i) = \sum_{l=1}^{m} \widehat{\boldsymbol{\beta}}_l h_l(\mathbf{x}_i) \tag{1}$$

where $\widehat{f}$ is the estimated function obtained from the calculation of the observed data points and $\widehat{\boldsymbol{\beta}} = \left(\widehat{\boldsymbol{\beta}}_1, \widehat{\boldsymbol{\beta}}_2, \ldots, \widehat{\boldsymbol{\beta}}_m\right)$ is the estimated coefficients vector. From the view of neural networks, the main difference between the classic regression and the neural networks is the structure of the $h(.)$ vector. In neural networks, $h(.)$ consists of the input values, weights and biases in neural networks. Actually $h(.)$ is the output of the activation function, which is the sum of the input weights and input values referring to explanatory variables and biases. Therefore, Eq. (1) can be rewritten in neural networks context as:

$$\widehat{f}(\mathbf{x}_i) = \sum_{l=1}^{m} \widehat{\boldsymbol{\beta}}_l h(\mathbf{w}_l.\mathbf{x}_i + b_l) \tag{2}$$

where $\mathbf{w}_l$ corresponds the input weight vector linking the $l$th hidden node and the input nodes, $b_l$ is the bias value of the $l$th hidden node and $\widehat{\boldsymbol{\beta}}_l$ is the output weight vector linking the $l$th hidden node and the output nodes. Here $m$ is the number of nodes in hidden layer and $h$ is the activation function. $\mathbf{w}_l.\mathbf{x}_i$ corresponds the inner product between $\mathbf{w}_l$ and $\mathbf{x}_i$ [3], [4]. By considering new parameters in estimation function given by Eq. (2), the problem can be defined in matrix form for SLFNs as follows [3], [4], [43]:

$$\begin{bmatrix} h(\mathbf{w}_1.\mathbf{x}_1 + b_1) & \ldots & h(\mathbf{w}_m.\mathbf{x}_1 + b_m) \\ \vdots & \ldots & \vdots \\ h(\mathbf{w}_1.\mathbf{x}_N + b_1) & \ldots & h(\mathbf{w}_m.\mathbf{x}_N + b_m) \end{bmatrix}_{Nxm} \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_m \end{bmatrix}_{mx1} = \begin{bmatrix} \mathbf{t}_1 \\ \vdots \\ \mathbf{t}_N \end{bmatrix}_{Nx1} \tag{3}$$

Eq. (3) can be written in matrix notation as

$$\mathbf{H}\boldsymbol{\beta} = \mathbf{T} \tag{4}$$

where $\mathbf{H}$ is called as the output matrix of hidden layer in the neural network by Huang et al. [3], [4]. For appropriate $\mathbf{w}_l$, $b_l$ and $\beta$ parameters, the solution of Eq. (4) is obtained by minimizing the following system in SLFNs:

$$\left\|\mathbf{H}\left(\widehat{\mathbf{w}}_1, \ldots, \widehat{\mathbf{w}}_m, \widehat{b}_1, \ldots, \widehat{b}_m\right)\widehat{\beta} - T\right\| = \min_{\mathbf{w}_l, b_l, \beta} \|\mathbf{H}\beta - T\|. \tag{5}$$

The solution of Eq. (5) is calculated by the least squares method as $\widehat{\boldsymbol{\beta}}_{ELM} = \left(\mathbf{H}^T\mathbf{H}\right)^{-1}\mathbf{H}^T\mathbf{T}$ unless $m \neq N$ and $\mathbf{H}^T\mathbf{H}$ is singular. Otherwise, the Moore–Penrose inverse of matrix $\mathbf{H}$, say $\mathbf{H}^+$, should be used to get the optimal solution as $\widehat{\boldsymbol{\beta}}_{ELM} = \mathbf{H}^+\mathbf{T}$.

Orthogonal projection method, iterative methods and singular value decomposition (SVD) are well-known and effective ways to calculate the Moore-Penrose inverse [60], [61]. According to the orthogonal projection method [61], the Moore–Penrose inverse of $\mathbf{H}$ is found as follows:

$$\mathbf{H}^+ = \begin{cases} \left(\mathbf{H}^T\mathbf{H}\right)^{-1}\mathbf{H}^T, & rank(\mathbf{H}) = m \\ \mathbf{H}^T\left(\mathbf{H}\mathbf{H}^T\right)^{-1}, & rank(\mathbf{H}) = N \end{cases} \tag{6}$$

The steps of ELM algorithm proposed by Huang et al. [4] can be summarised in Algorithm 1. Besides, the visual representation of the algorithm is given in Figure 1:

**Algorithm 1** ELM Algorithm

1: Randomly assign input weights and bias parameters.
2: Calculate the hidden layer output matrix $\mathbf{H}$.
3: Calculate the output weights via $\widehat{\boldsymbol{\beta}}_{ELM} = \mathbf{H}^+\mathbf{T}$ by using Eq. (6).

When there is a multicollinearity problem, inverting the matrix $\left(\mathbf{H}^T\mathbf{H}\right)^{-1}$ can sometimes be impossible and sometimes unstable. Therefore, alternative methods to ordinary ELM are recommended. On the other hand, ordinary ELM does not have sparsity property; that is, does not do variable selection. The methods proposed on the basis of ELM as a solution to these two problems are as follows:

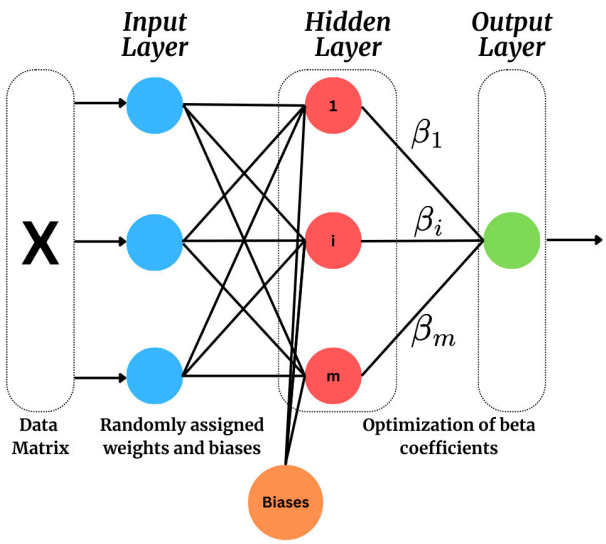



**FIGURE 1.** The general representation of the basic ELM model.

Reference [48] Li and Niu considered ridge regression in the context of ELM which was originally proposed by Hoerl and Kennard [41] and defined the Ridge-ELM as

$$\hat{\boldsymbol{\beta}}^{k}_{Ridge-ELM}=\left(\mathbf{H}^T\mathbf{H}+k\mathbf{I}_m\right)^{-1}\mathbf{H}^T\mathbf{Y},\ k\geq \mathbf{0}$$

where $k$ is the ridge tuning parameter and $\mathbf{I}_\varphi$ is the identity matrix with dimension $\varphi$. Although the value of $k$ affects the performance of Ridge-ELM, there is no single best and suitable method for selecting the ridge tuning parameter. Regression studies literatures [48] and [62] indicates that Ridge-ELM can provide smaller error than ELM if $k$ is correctly determined.

Yıldırım and Özkale [53] considered the Liu regression in the context of ELM which was originally defined by Liu [42] and defined the Liu-ELM as

$$\hat{\boldsymbol{\beta}}^{d}_{Liu-ELM}=\left(\mathbf{H}^T\mathbf{H}+\mathbf{I}_m\right)^{-1}\left(\mathbf{H}^T\mathbf{Y}+d\hat{\boldsymbol{\beta}}_{ELM}\right) \tag{7}$$

where $0 < d < 1$ is the Liu tuning parameter which shrinks each element of $\hat{\boldsymbol{\beta}}_{ELM}$ by the same $d$ value. Since $\hat{\boldsymbol{\beta}}^{d}_{Liu-ELM}$ is in linear form of $d$, computing $\hat{\boldsymbol{\beta}}^{d}_{Liu-ELM}$ is much easier and faster than $\hat{\boldsymbol{\beta}}^{k}_{Ridge-ELM}$ which provides computationally effective and cost minimized solutions for machine learning. $\hat{\boldsymbol{\beta}}^{d}_{Liu-ELM}$ is also a convex combination of $\hat{\boldsymbol{\beta}}_{ELM}$ and $\hat{\boldsymbol{\beta}}^{k}_{Ridge-ELM}$ for $k=1$ which claims that $\hat{\boldsymbol{\beta}}^{d}_{Liu-ELM}$ provides solution paths between $\hat{\boldsymbol{\beta}}_{ELM}$ and $\hat{\boldsymbol{\beta}}^{(k=1)}_{Ridge-ELM}$ as $d$ goes from 1 to 0:

$$\hat{\boldsymbol{\beta}}^{d}_{Liu-ELM}=d\hat{\boldsymbol{\beta}}_{ELM}+(1-d)\hat{\boldsymbol{\beta}}^{(k=1)}_{Ridge-ELM}.$$

Yıldırım and Özkale [57] proposed OK-ELM which takes the advantages of ridge and Liu regressions as

$$\widehat{\boldsymbol{\beta}}_{OK-ELM}=\left(\mathbf{H}^T\mathbf{H}+k\mathbf{I}_\mathbf{m}\right)^{-1}\left(\mathbf{H}^T\mathbf{Y}+kd\hat{\boldsymbol{\beta}}_{ELM}\right)$$

where $0 < d < 1$ and $k > 0$ are the tuning parameters. OK-ELM is a convex combination of Ridge-ELM and ELM:

$$\widehat{\boldsymbol{\beta}}_{OK-ELM}=d\hat{\boldsymbol{\beta}}_{ELM}+(1-d)\,\widehat{\boldsymbol{\beta}}^{(k)}_{Ridge-ELM},$$

where $d$ serves as a balance parameter between ELM and Ridge-ELM and refers the relative contributions of them. Basically, while the higher $d$ towards to 1 yields more contribution in favor of ELM, the lower $d$ increases the effect of Ridge-ELM to the solution.

## III. METHODOLOGY

### A. THE PROPOSED ALGORITHMS

The ridge and Liu estimator stand-alone is directly relying on classical least squares and therefore has the tendency to yield inadequate or misleading information in dealing with multicollinearity. Therefore, two-parameter estimators have come to the prominence in order to retain the existing capabilities and make more accurate estimates through holding the estimators in a simultaneous model. Although these estimators possess the skills of the two estimators to an extent in terms of the mathematical structure, yet there is no single superior model in real-life applications. Motivated by this point, we propose the following two-parameters ELM (named as TP1-ELM) algorithm by incorporating the k-d estimator [58] from the two-parameter estimators into the ELM algorithm:

$$\widehat{\boldsymbol{\beta}}_{TP1-ELM}(k,d)=\left(\mathbf{H}^T\mathbf{H}+k\mathbf{I}_m\right)^{-1}\left(\mathbf{H}^T\mathbf{Y}+d\hat{\boldsymbol{\beta}}_{Ridge-ELM}\right) \tag{8}$$

where $-\infty < d < \infty$ and $k > 0$ are the tuning parameters. The proposed TP1-ELM algorithm aims to eliminate the stability problem in the classical least squares by incorporating Liu and Ridge approaches simultaneously. As a result of this property, the ridge or Liu estimator is considered to outperform the standalone ridge or standalone Liu estimator by adjusting the parameters in a tradeoff by making the parameter selection more flexible. The TP2-ELM algorithm is a general implementation of the classical least squares, ridge and Liu estimators, which can be derived in special cases:

$$\begin{aligned}\widehat{\beta}(0,1)&=\widehat{\beta}_{ELM}\\ \widehat{\beta}(k,1-k)&=\widehat{\beta}_{Ridge-ELM}\\ \widehat{\beta}(0,d)&=\widehat{\beta}_{Liu-ELM},0\leq d\leq 1\end{aligned}$$

An alternative two-parameters estimator originally introduced by Yang and Chang [59] can be proposed for first utilization in the field of ELM named as TP2-ELM and defined as follows:

$$\begin{aligned}&\widehat{\boldsymbol{\beta}}_{TP2-ELM}(k,d)\\ &=\left(\mathbf{H}^T\mathbf{H}+\mathbf{I}_m\right)^{-1}\left(\mathbf{H}^T\mathbf{H}+d\mathbf{I}_m\right)\left(\mathbf{H}^T\mathbf{H}+k\mathbf{I}_m\right)^{-1}\left(\mathbf{H}^T\mathbf{Y}\right)\end{aligned} \tag{9}$$

where $0 < d < 1$ and $k > 0$ are the tuning parameters. Analogously to the TP1-ELM algorithm, the OLS, ridge and

Liu estimators can be extracted based on specific selections of the parameters:

$$\widehat{\beta}(0,1) = \widehat{\beta}_{ELM}$$
$$\widehat{\beta}(k,1) = \widehat{\beta}_{Ridge-ELM}$$
$$\widehat{\beta}(0,d) = \widehat{\beta}_{Liu-ELM}, 0 \leq d \leq 1$$

In the parameter selection of the proposed algorithms, a grid search via cross-validation approach within a defined value range or the analytical selection methods proposed by Sakallıoğlu and Kaçranlar [58] and Yang and Chang [59] can be implemented. A pseudo code implementation of the proposed TP1-ELM and TP2-ELM algorithms is presented in Algorithm 2.

**Algorithm 2** Pseudo Code for the Proposed Algorithms

**Input:** Generate the input layer weights $\widehat{\mathbf{w}}_1, \ldots, \widehat{\mathbf{w}}_m$ and biases $\widehat{b}_1, \ldots, \widehat{b}_m$ based on a certain number of the hidden layer neurons ($m$), $k$ *range* and $d$ *range* for the tuning parameters space, $\varphi$ as the trial number.
**Output:** The $\beta$ coefficient vector.
Calculate the hidden layer output matrix H based on inputs.
Set the seed value for reproducibility
**for** $trial = 1 : \varphi$ **do**
  **for** $k$ *in* $k$ *range* **do**
    **for** $d$ *in* $d$ *range* **do**
      Obtain TP1-ELM coefficient vector via Eq. 8
      Obtain TP2-ELM coefficient vector via Eq. 9
      Calculate performance metrics for CV split
    **end for**
  **end for**
**end for**
Determine the parameters with the lowest mean error values across the trials.
Compute the error scores on the test data with the optimal parameters

### B. DATA SOURCE

In this study, we utilized seven datasets with structural and qualitative diversity across various domains to compare the performance of the statistical and machine learning models under consideration. These datasets were sourced from the UCI Machine Learning Repository [63], Kaggle platform [64], several universites databases [65], [66]. The specifications of the data sets and target variables covered in the study can be briefly summarized as follows: Auto Price data to predict vehicle prices, Boston Housing data to predict the median value of a real estate price, Fish data to predict quantitative acute aquatic toxicity, Forests data to predict the burned area of forest fires, Machine CPU data to predict relative cpu performance, Servo data to predict the rise time of a servomechanism, Slump data to predict concrete slump and Strikes data to predict the level of strike volume (days lost due to industrial disputes per 1000 wage salary earners). The details of dimensions are listed in Table 1.

### C. EXPERIMENTAL SETTINGS

In the process of benchmarking model performances, experimental settings are crucial for ensuring reproducibility. In this context, the following steps were followed in training and testing statistical models and machine learning algorithms:

- Prior to training, the data is pre-processed by standardizing it to have a mean of zero and unit variance, using the following formula:
$$\mathbf{x} = \left(\frac{x_i - \overline{x}}{sd(\mathbf{x})}\right).$$
- The dataset is partitioned into three-fourths for training and the remaining one-fourth for testing.
- Models are developed using five-times repeated five-fold cross-validation on the training data, and their generalization performances are evaluated on the test data.
- To mitigate the effects of randomness, thirty trials are conducted, and the root mean square error (RMSE) is calculated for each trial. Mean values are reported using the following equation:
$$RMSE = \sqrt{\frac{1}{N}\sum_{j=1}^{N}\left(\mathbf{o}_j - \mathbf{t}_j\right)^2}$$
where $\left(\mathbf{o}_j - \mathbf{t}_j\right)$ corresponds to the error between the actual and output values of the target variable. RMSE is defined in the interval $[0, \infty]$ and the lower the value of this criterion, the better the model performance.
- The number of hidden layer nodes for ELM-based models is kept constant at 100.
- A grid search approach is employed to assess the tuning parameters of the ELM-based statistical models. The *delta* for Ridge-ELM, $d$ for Liu-ELM, and both *delta* and $d$ parameters for OK-ELM, TP1-ELM, and TP2-ELM models are calculated via cross-validation, considering the following ranges:
$$k \in \left[10^{-5}, 10^{-4}, .., 10^{-2}, 0.02, \ldots, 0.1, 0.2, \ldots, 1, 1.5, 2, \ldots, 5\right]$$
$$d \in \left[10^{-5}, 10^{-4}, .., 10^{-2}, 0.02, 0.03 \ldots, 0.1, 0.15, 0.2, \ldots, 1\right].$$
- Optimal parameter determination for machine learning algorithms is similarly conducted by analyzing a space of thirty possible parameters for each model based on cross-validation data. The parameters yielding the lowest error value are then assigned.
- The reduction rate (RR), representing the relative improvement of the proposed algorithms over other algorithms, is evaluated in percentage terms using the following equation for clearer interpretation:
$$RR = \frac{(any\ algorithm) - \left(TP_{1,2} - ELM\right)}{(any\ algorithm)} \times 100$$
- The details of this process are given as a visual representation in Figure 2.

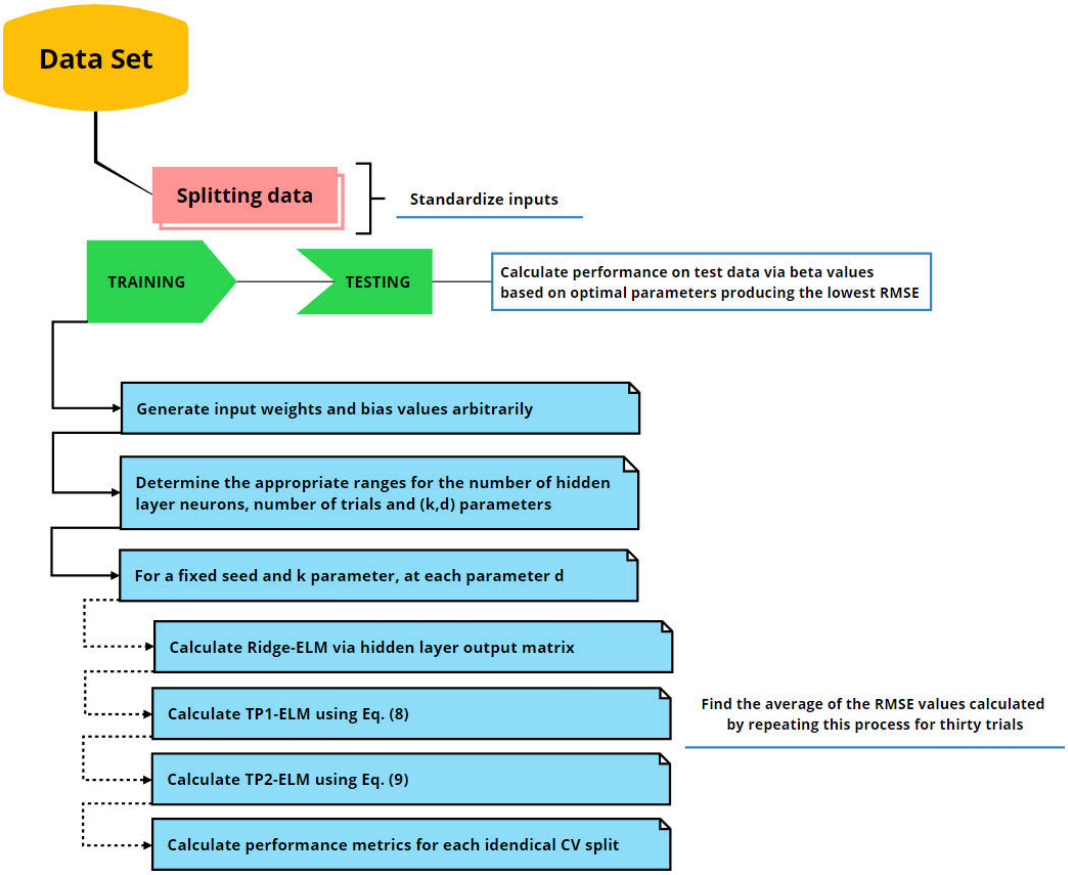



**FIGURE 2.** The flow chart of the calculation for the TP-ELMs.

**TABLE 1.** The properties of data sets used in this study.

| Data sets | Sample Size | Attributes |
|---|---|---|
| Auto Price | 159 | 13 |
| Boston Housing | 506 | 14 |
| Fish | 908 | 7 |
| Forest | 517 | 11 |
| Machine CPU | 209 | 7 |
| Servo | 167 | 5 |
| Strikes | 625 | 5 |

### D. THE ASSESSMENT OF MULTICOLLINEARITY

The datasets were examined for multicollinearity using the Variance Inflation Factor (VIF) and eigenvalue analysis criteria, derived from the following equations:

$$VIF_j = C_{ij} = (1 - R_j^2)^{-1}$$

where $C = (X'X)^{-1}$ is the coefficient of determination calculated via the regression fit of $x_j$ over the rest p-1 variables. In the dataset, the presence of a VIF value greater than 5 belonging to the one of the variables indicates a strong multicollinearity problem [67].

As an effective and widely used method in addition to the VIF approach, the eigenvalue analysis is based on the decomposition of the X'X matrix into eigenvalues and eigenvectors and calculated as follows:

$$X'X = T\Lambda T'$$

where $\Lambda_{p\times p}$ is the diagonal matrix, whose diagonal elements correspond to the eigenvalues ($\lambda_i, i = 1, 2, \ldots, p$) and $T_{p\times p}$ is the orthogonal matrix whose columns correspond to the eigenvectors of $X'X$ matrix.Small-valued eigenvalues can be indicative of the presence of multicollinearity across columns of data. Rather than focusing on each eigenvalue, the condition number (CN), which is basically a representation of the spread of eigenvalues, is commonly used and is calculated as follows:

$$CN = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Typically, a Condition Number (CN) exceeding 1000 is considered evidence of severe multicollinearity, while a CN between 100 and 1000 implies strong multicollinearity between the columns (i.e., variables) of the data matrix [67].

The VIF, eigenvalue, and condition number criteria were calculated for the seven datasets in the study, and the results are reported in Table 2. According to the findings, the Auto Price and Slump datasets exhibit multicollinearity issues. Additionally, the Boston Housing dataset shows

**TABLE 2.** The results of multicollinearity diagnostics for each data set.

| Data set | Criterion | Variables ID 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Auto Price | VIF | 6,1088 | 7,7759 | 5,6902 | 2,2876 | 16,0658 | 9,0702 | 2,2690 | 1,4417 | 2,5073 | 7,1937 | 1,8893 | 6,1194 | |
| | Eigenvalue | 6,2844 | 1,8772 | 1,2903 | 0,8847 | 0,5191 | 0,3635 | 0,2743 | 0,1444 | 0,1240 | 0,1043 | 0,0871 | 0,0466 | |
| | CN | 134,7581 | | | | | | | | | | | | |
| Boston Housing | VIF | 1,7922 | 2,2988 | 3,9916 | 1,0740 | 4,3937 | 1,9337 | 3,1008 | 3,9559 | 7,4845 | 9,0086 | 1,7991 | 1,3485 | 2,9415 |
| | Eigenvalue | 6,1268 | 1,4333 | 1,2426 | 0,8576 | 0,8348 | 0,6574 | 0,5354 | 0,3961 | 0,2769 | 0,2202 | 0,1860 | 0,1693 | 0,0635 |
| | CN | 96,4717 | | | | | | | | | | | | |
| Fish | VIF | 2,1315 | 1,3977 | 1,6058 | 1,0786 | 1,2319 | 2,3537 | | | | | | | |
| | Eigenvalue | 1,6422 | 1,4729 | 1,1253 | 0,9636 | 0,5916 | 0,2043 | | | | | | | |
| | CN | 8,0382 | | | | | | | | | | | | |
| Forests | VIF | 1,4334 | 1,4456 | 1,6981 | 2,3616 | 2,1259 | 1,5794 | 2,6667 | 1,9138 | 1,1428 | 1,0478 | | | |
| | Eigenvalue | 2,8597 | 1,5594 | 1,2961 | 1,2125 | 0,9315 | 0,6926 | 0,4756 | 0,4631 | 0,2947 | 0,2148 | | | |
| | CN | 13,3122 | | | | | | | | | | | | |
| Machine CPU | VIF | 1,2016 | 2,9020 | 3,2740 | 1,8583 | 1,9662 | 1,8914 | | | | | | | |
| | Eigenvalue | 3,3567 | 0,8294 | 0,7392 | 0,4963 | 0,4044 | 0,1739 | | | | | | | |
| | CN | 19,3004 | | | | | | | | | | | | |
| Slump | VIF | 2,5147 | 88,6201 | 100,2531 | 110,9583 | 54,2953 | 2,3860 | 156,4193 | 84,3636 | 7,4860 | 10,0768 | | | |
| | Eigenvalue | 3,0246 | 2,0919 | 1,1848 | 1,1284 | 0,8833 | 0,6927 | 0,6329 | 0,3021 | 0,0576 | 0,0017 | | | |
| | CN | 1784,4887 | | | | | | | | | | | | |
| Strikes | VIF | 1,0557 | 1,0572 | 1,0556 | 1,0021 | | | | | | | | | |
| | Eigenvalue | 1,3522 | 1,0073 | 0,8239 | 0,8166 | | | | | | | | | |
| | CN | 1,6559 | | | | | | | | | | | | |

a borderline condition number value, indicating potential multicollinearity concerns.

### E. THE LIST OF MACHINE LEARNING ALGORITHMS

The machine learning models evaluated in this study can be categorized into five subgroups:

- Splines-based models: MARS (Multivariate Adaptive Regression Splines), Linear Regresssion (LR), Rulefit and Partial Least Sqaures (PLS) belong to this category.
- Kernel-based models: Support Vector Machines (SVM) fall into this category.
- Tree-based models: These include Classification and Regression Trees (CART), Bagging with different base learners including CART, MARS, MLP etc., Random Forests (RF), LightGBM and Extreme Gradient Boosting.
- Instance-based models: K-Nearest Neighbors (KNN) is an example of this subgroup. There are ensemble models incorporating different regression (e.g. stepwise regression) [68] or classification approaches [69], [70] within the KNN algorithm, but the classical KNN algorithm is employed in this study.
- Neural Networks-Based models: Multilayer Perceptron (MLP) and Bayesian Additive Trees (BAT) are representative of this subgroup.

## IV. RESULTS AND DISCUSSION

In this section, we present detailed performance comparison results of both statistical models and machine learning algorithms. A comprehensive analysis of the ELM-based statistical models, including the two newly proposed algorithms, is provided separately, both among themselves and in comparison to machine learning algorithms.

### A. THE PERFORMANCE COMPARISON OF ELM-BASED ALGORITHMS AND THE PROPOSED ALGORITHMS

Initially, comprehensive comparison results of the performance of the proposed TP1-ELM and TP2-ELM algorithms with Ridge-ELM, Liu-ELM, and OK-ELM on both training and test data are presented in Table 3. A visual comparison of performance using calculated reduction rate values, based on the results in Table 3, is shown in Figure 3. Due to large variations in scale, percentages based on ELM performance are not included in Figure 3 but are discussed in the text to enhance the interpretability and readability of visual comparisons. The following conclusions can be drawn from Table 3 and Figure 3:

- The training performances of TP1-ELM and TP2-ELM are competitive with other algorithms. At least one of the proposed algorithms shows

**TABLE 3.** Performance comparisons of each statistical algorithms based on RMSE criterion.

| Dataset | Model | d | delta | RMSE (Train) | SD | RMSE (Test) | SD |
|---|---|---|---|---|---|---|---|
| Auto Price | ELM | * | * | 0,0671 | 0,0094 | 1,6947 | 0,5818 |
| | Ridge-ELM | * | 1,96 | 0,2967 | 0,0112 | 0,4576 | 0,0302 |
| | Liu-ELM | 0,01208 | * | 0,2943 | 0,0116 | 0,4710 | 0,0338 |
| | OK-ELM | 0,00624 | 1,96 | 0,2951 | 0,0111 | 0,4573 | 0,0311 |
| | TP1-ELM | 0,40804 | 2,24 | 0,2912 | 0,0115 | 0,4465 | 0,0318 |
| | TP2-ELM | 0,84 | 1,82 | 0,2917 | 0,0110 | 0,4274 | 0,0280 |
| Boston Housing | ELM | * | * | 0,2869 | 0,0131 | 0,4481 | 0,0481 |
| | Ridge-ELM | * | 0,26 | 0,3141 | 0,0111 | 0,4128 | 0,0297 |
| | Liu-ELM | 0,39 | * | 0,3140 | 0,0112 | 0,4119 | 0,0300 |
| | OK-ELM | 0,258 | 1,54 | 0,3153 | 0,0112 | 0,4088 | 0,0295 |
| | TP1-ELM | 0,224 | 0,4 | 0,3152 | 0,0111 | 0,4027 | 0,0283 |
| | TP2-ELM | 0,65 | 0,0762 | 0,3123 | 0,0112 | 0,4101 | 0,0273 |
| Fish | ELM | * | * | 0,5167 | 0,0072 | 0,6987 | 0,0454 |
| | Ridge-ELM | * | 2,18 | 0,5761 | 0,0047 | 0,6148 | 0,0088 |
| | Liu-ELM | 0,08002 | * | 0,5625 | 0,0044 | 0,6170 | 0,0101 |
| | OK-ELM | 0,114 | 2,88 | 0,5684 | 0,0045 | 0,6137 | 0,0092 |
| | TP1-ELM | 0,21604 | 2,38 | 0,5761 | 0,0047 | 0,6148 | 0,0089 |
| | TP2-ELM | 0,49 | 1,904002 | 0,5761 | 0,0048 | 0,6145 | 0,0087 |
| Forests | ELM | * | * | 0,8613 | 0,0110 | 1,1832 | 0,0736 |
| | Ridge-ELM | * | 4,8 | 0,9543 | 0,0046 | 1,0214 | 0,0093 |
| | Liu-ELM | 0,0001 | * | 0,9101 | 0,0071 | 1,0294 | 0,0184 |
| | OK-ELM | 0,0001 | 4,8 | 0,9443 | 0,0046 | 1,0714 | 0,0073 |
| | TP1-ELM | 0,17008 | 4,9 | 0,9421 | 0,0036 | 1,0414 | 0,0065 |
| | TP2-ELM | 0,0001 | 4,3 | 0,9334 | 0,0031 | 1,0099 | 0,0063 |
| Machine CPU | ELM | * | * | 0,1019 | 0,0029 | 0,7533 | 0,2449 |
| | Ridge-ELM | * | 0,0642 | 0,1508 | 0,0039 | 0,3189 | 0,0501 |
| | Liu-ELM | 0,15002 | * | 0,2261 | 0,0073 | 0,3501 | 0,0558 |
| | OK-ELM | 0,08602 | 0,166 | 0,1644 | 0,0044 | 0,3114 | 0,0486 |
| | TP1-ELM | 0,22202 | 0,1642 | 0,1591 | 0,0043 | 0,3149 | 0,0499 |
| | TP2-ELM | 0,72 | 0,02222 | 0,1640 | 0,0041 | 0,3068 | 0,0490 |
| Slump | ELM | * | * | 0,0611 | 0,0098 | 0,3321 | 0,1460 |
| | Ridge-ELM | * | 0,632 | 0,1672 | 0,0155 | 0,3019 | 0,0450 |
| | Liu-ELM | 0,138 | * | 0,1878 | 0,0138 | 0,3089 | 0,0406 |
| | OK-ELM | 0,08204 | 0,714 | 0,1616 | 0,0143 | 0,2982 | 0,0471 |
| | TP1-ELM | 0,0302 | 0,658 | 0,1669 | 0,0156 | 0,2619 | 0,0353 |
| | TP2-ELM | 0,64 | 0,4222 | 0,1613 | 0,0131 | 0,2981 | 0,0466 |
| Strikes | ELM | * | * | 0,8272 | 0,0039 | 1,0840 | 0,2298 |
| | Ridge-ELM | * | 0,108 | 0,8626 | 0,0038 | 0,8968 | 0,0096 |
| | Liu-ELM | 0,3 | * | 0,8735 | 0,0030 | 0,9064 | 0,0124 |
| | OK-ELM | 0,17402 | 0,13 | 0,8573 | 0,0034 | 0,8923 | 0,0094 |
| | TP1-ELM | 0,19044 | 0,208 | 0,8617 | 0,0038 | 0,8666 | 0,0066 |
| | TP2-ELM | 0,68 | 0,0046 | 0,8568 | 0,0033 | 0,8714 | 0,0105 |

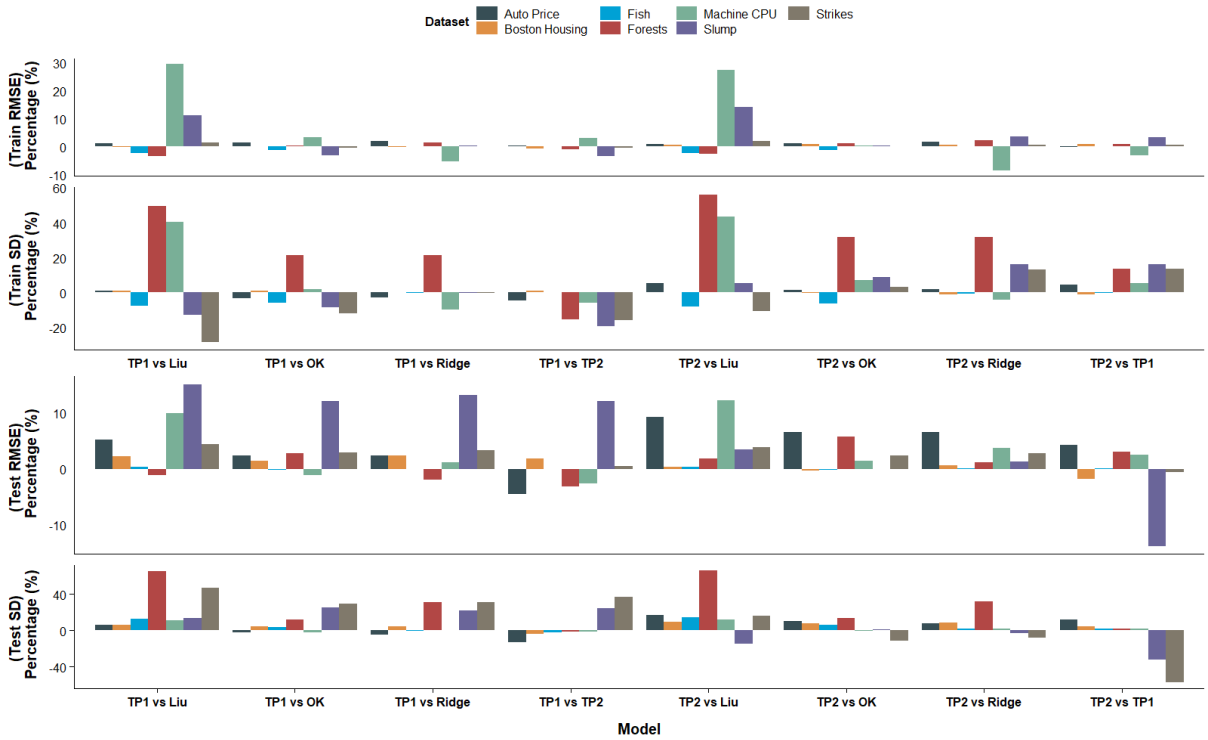



**FIGURE 3.** The overall comparison of ELM-based statistical models for both train and testing performance based on reduction rates.

**TABLE 4.** Coefficient norm comparison for each statistical algorithm.

| Dataset | Model | Norm | SD | Dataset | Model | Norm | SD |
|---|---|---|---|---|---|---|---|
| Auto Price | ELM | 57,7752 | 10,4343 | Machine CPU | ELM | 57,1196 | 7,0261 |
| | Ridge-ELM | 2,5481 | 0,0905 | | Ridge-ELM | 9,5113 | 0,4728 |
| | Liu-ELM | 2,5706 | 0,1341 | | Liu-ELM | 9,8764 | 1,0567 |
| | OK-ELM | 2,6324 | 0,0897 | | OK-ELM | 9,0178 | 0,6254 |
| | TP1-ELM | 2,7082 | 0,1029 | | TP1-ELM | 9,3592 | 0,4686 |
| | TP2-ELM | 2,6240 | 0,0820 | | TP2-ELM | 10,0332 | 0,5347 |
| Boston Housing | ELM | 12,3740 | 1,2719 | Slump | ELM | 12,8130 | 2,3867 |
| | Ridge-ELM | 6,0931 | 0,3340 | | Ridge-ELM | 2,7237 | 0,1397 |
| | Liu-ELM | 6,5575 | 0,4605 | | Liu-ELM | 2,8096 | 0,2668 |
| | OK-ELM | 6,5181 | 0,4611 | | OK-ELM | 3,0404 | 0,2132 |
| | TP1-ELM | 6,0056 | 0,3305 | | TP1-ELM | 2,7196 | 0,1394 |
| | TP2-ELM | 6,3418 | 0,3818 | | TP2-ELM | 3,0155 | 0,1946 |
| Fish | ELM | 40,6888 | 6,1286 | Strikes | ELM | 171,8604 | 23,2722 |
| | Ridge-ELM | 3,4479 | 0,1491 | | Ridge-ELM | 26,3884 | 1,0555 |
| | Liu-ELM | 5,7294 | 0,4603 | | Liu-ELM | 52,6346 | 6,9376 |
| | OK-ELM | 6,0678 | 0,5996 | | OK-ELM | 42,3485 | 3,9297 |
| | TP1-ELM | 3,4178 | 0,1470 | | TP1-ELM | 27,0780 | 1,0861 |
| | TP2-ELM | 4,0299 | 0,2216 | | TP2-ELM | 38,5599 | 2,1897 |
| Forests | ELM | 18,8127 | 2,0581 | | | | |
| | Ridge-ELM | 1,6162 | 0,0923 | | | | |
| | Liu-ELM | 3,9754 | 0,1841 | | | | |
| | OK-ELM | 1,5970 | 0,0829 | | | | |
| | TP1-ELM | 1,5907 | 0,0823 | | | | |
| | TP2-ELM | 1,2163 | 0,0786 | | | | |

considerable superiority (improvement rates of up to 29.64%) over Liu-ELM in most of the data (including auto-price, boston housing, machine cpu, slump and strikes) and the difference is especially obvious in the Slump and Machine CPU datasets. However, the basic ELM algorithm outperforms all models in terms of training performance, consistent with expectations.

- Regarding deviation scores on the training data, at least one of the proposed algorithms displays greater stability on most datasets to some extent (up to 56.22%).
- In terms of generalization performance, a primary focus of this study, the proposed algorithms clearly outperform other models on almost all datasets (up to 15.22%). Although they show slight weakness on Forest and Machine CPU datasets, the scores remain highly competitive. Notably, TP1-ELM and TP2-ELM significantly outperform the ELM algorithm, with TP1-ELM surpassing it by up to 73.65% and TP2-ELM by up to 74.78%.
- Similarly, standard deviations of test performance suggest that the proposed algorithms exhibit much lower variability (up to 65.61%), indicating more stable and generalizable results across almost all datasets.
- While the proposed algorithms generally perform comparably, TP2-ELM demonstrates slightly superior testing performance compared to TP1-ELM.

The norm means and deviations of coefficient estimates for the ELM-based statistical models and proposed algorithms are calculated and presented in Table 4, providing insights into coefficient volatility. Results indicate that the TP1-ELM algorithm yields smaller and more stable norm values than all other algorithms on Boston, Fish, and Slump datasets. However, TP2-ELM outperforms on the Forest dataset. Conversely, Ridge-ELM performs slightly better on Auto Price and Strikes datasets, while OK-ELM shows superiority on Machine CPU data.

### B. THE PERFORMANCE COMPARISON OF MACHINE LEARNING ALGORITHMS AND THE PROPOSED ALGORITHMS

The performance of the proposed algorithms, along with seventeen widely used machine learning algorithms, was

**TABLE 5.** Performance results of machine learning algorithms for both training and testing data sets.

| Data Set | Split | Model | Bag-Cart | Bag-Mars | Bag-MLP | BAT | LightGBM | XGBoost | Cart | KNN | LR | MARS | MLP | PLS | RF | RuleFit | SMV-Linear | SVM-Poly | SVM-Radial |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Auto Price | Train | RMSE | 0,3666 | 0,3698 | 0,3926 | 0,3410 | 0,3639 | 0,3001 | 0,3152 | 0,2790 | 0,3748 | 0,3545 | 0,3315 | 0,3082 | 0,2819 | 0,2720 | 0,3078 | 0,3679 | 0,3671 |
| | | SD | 0,0698 | 0,0553 | 0,0414 | 0,0847 | 0,1670 | 0,2510 | 0,0876 | 0,0399 | 0,0442 | 0,1090 | 0,2270 | 0,0212 | 0,0523 | 0,1540 | 0,0673 | 0,2050 | 0,2048 |
| | Test | RMSE | 0,3777 | 0,5082 | 0,4514 | 0,3974 | 0,3989 | 0,5609 | 0,3930 | 0,3526 | 0,4912 | 0,4533 | 0,4468 | 0,4555 | 0,3592 | 0,4523 | 0,4489 | 0,4540 | 0,4260 |
| Boston Housing | Train | RMSE | 0,4223 | 0,4207 | 0,3942 | 0,3722 | 0,3963 | 0,3295 | 0,3285 | 0,2944 | 0,4644 | 0,3250 | 0,3293 | 0,4226 | 0,3270 | 0,2992 | 0,3242 | 0,4649 | 0,3950 |
| | | SD | 0,0669 | 0,0726 | 0,0260 | 0,0536 | 0,2330 | 0,1180 | 0,0533 | 0,0487 | 0,0529 | 0,0931 | 0,0559 | 0,0418 | 0,0294 | 0,0957 | 0,0447 | 0,1350 | 0,1255 |
| | Test | RMSE | 0,4678 | 0,4556 | 0,4311 | 0,4159 | 0,4161 | 0,3782 | 0,5419 | 0,4388 | 0,5722 | 0,4618 | 0,3964 | 0,5655 | 0,3932 | 0,3833 | 0,5595 | 0,5612 | 0,4571 |
| Fish | Train | RMSE | 0,6194 | 0,5644 | 0,6170 | 0,6272 | 0,6253 | 0,5722 | 0,5809 | 0,5662 | 0,6189 | 0,6181 | 0,5964 | 0,6361 | 0,5600 | 0,5692 | 0,5725 | 0,6191 | 0,6192 |
| | | SD | 0,0486 | 0,0455 | 0,0103 | 0,0337 | 0,1477 | 0,0652 | 0,0430 | 0,0474 | 0,0849 | 0,0363 | 0,3037 | 0,0290 | 0,0095 | 0,1118 | 0,0185 | 0,1064 | 0,1062 |
| | Test | RMSE | 0,6447 | 0,6609 | 0,7072 | 0,6154 | 0,6142 | 0,6145 | 0,7177 | 0,6129 | 0,6686 | 0,6801 | 0,6383 | 0,6685 | 0,6200 | 0,6303 | 0,6699 | 0,6704 | 0,6534 |
| Forest | Train | RMSE | 0,9886 | 0,9884 | 1,0002 | 0,9876 | 0,9853 | 0,9853 | 0,9853 | 1,0193 | 1,0365 | 0,9921 | 1,0262 | 0,9855 | 1,0026 | 0,9853 | 1,0306 | 1,0307 | 1,0307 |
| | | SD | 0,0200 | 0,0184 | 0,1489 | 0,0790 | 0,0326 | 0,0740 | 0,0850 | 0,0997 | 0,4852 | 0,0148 | 1,4575 | 0,0415 | 0,0090 | 0,2615 | 0,0050 | 0,0531 | 0,0534 |
| | Test | RMSE | 1,0310 | 1,0848 | 1,0307 | 1,1207 | 1,1417 | 1,0978 | 1,0282 | 1,0265 | 1,0966 | 1,0307 | 1,0350 | 1,0403 | 1,0437 | 1,0307 | 1,1092 | 1,0994 | 1,0675 |
| Machine CPU | Train | RMSE | 0,3503 | 0,3649 | 0,4242 | 0,3194 | 0,3669 | 0,2619 | 0,4343 | 0,3934 | 0,3549 | 0,3099 | 0,3759 | 0,2516 | 0,3971 | 0,3171 | 0,4166 | 0,3528 | 0,3615 |
| | | SD | 0,0573 | 0,0401 | 0,0327 | 0,0325 | 0,1120 | 0,1095 | 0,0686 | 0,0250 | 0,0690 | 0,0292 | 0,2140 | 0,0349 | 0,0392 | 0,0907 | 0,0623 | 0,1083 | 0,1109 |
| | Test | RMSE | 0,4256 | 0,5501 | 0,5893 | 0,4153 | 0,5099 | 0,4901 | 0,5434 | 0,3988 | 0,4963 | 0,4090 | 0,3944 | 0,4765 | 0,4072 | 0,5997 | 0,6277 | 0,4226 | 0,4228 |
| Slump | Train | RMSE | 0,3266 | 0,3032 | 0,2370 | 0,3207 | 0,3647 | 0,2919 | 0,1503 | 0,4248 | 0,3478 | 0,3264 | 0,1400 | 0,3061 | 0,1774 | 0,2583 | 0,3531 | 0,3335 | 0,2831 |
| | | SD | 0,0763 | 0,2263 | 0,0434 | 0,2181 | 0,1748 | 0,2039 | 0,0493 | 0,0546 | 0,0299 | 0,2085 | 0,1161 | 0,1263 | 0,0850 | 0,0595 | 0,2248 | 0,0902 | 0,0885 |
| | Test | RMSE | 0,4606 | 0,5912 | 0,5120 | 0,3426 | 0,4753 | 0,4554 | 0,5665 | 0,6048 | 0,3853 | 0,3609 | 0,4461 | 0,3717 | 0,4699 | 0,3936 | 0,3614 | 0,5186 | 0,3295 |
| Strikes | Train | RMSE | 0,7580 | 0,8031 | 0,7787 | 0,8250 | 0,7621 | 0,7107 | 0,8421 | 0,7763 | 0,9601 | 0,8456 | 0,8537 | 0,9126 | 0,7578 | 0,6955 | 0,9304 | 0,9542 | 0,9508 |
| | | SD | 0,0463 | 0,0501 | 0,4255 | 0,0458 | 0,0827 | 0,0661 | 0,0309 | 0,0599 | 0,1101 | 0,0497 | 1,0372 | 0,0043 | 0,0202 | 0,1081 | 0,0089 | 0,0095 | 0,0063 |
| | Test | RMSE | 0,9579 | 0,8809 | 1,0061 | 0,9485 | 0,9566 | 0,9345 | 1,0224 | 0,8894 | 0,9935 | 0,9563 | 0,8834 | 0,9917 | 0,8086 | 0,8002 | 1,0073 | 1,0173 | 0,9774 |

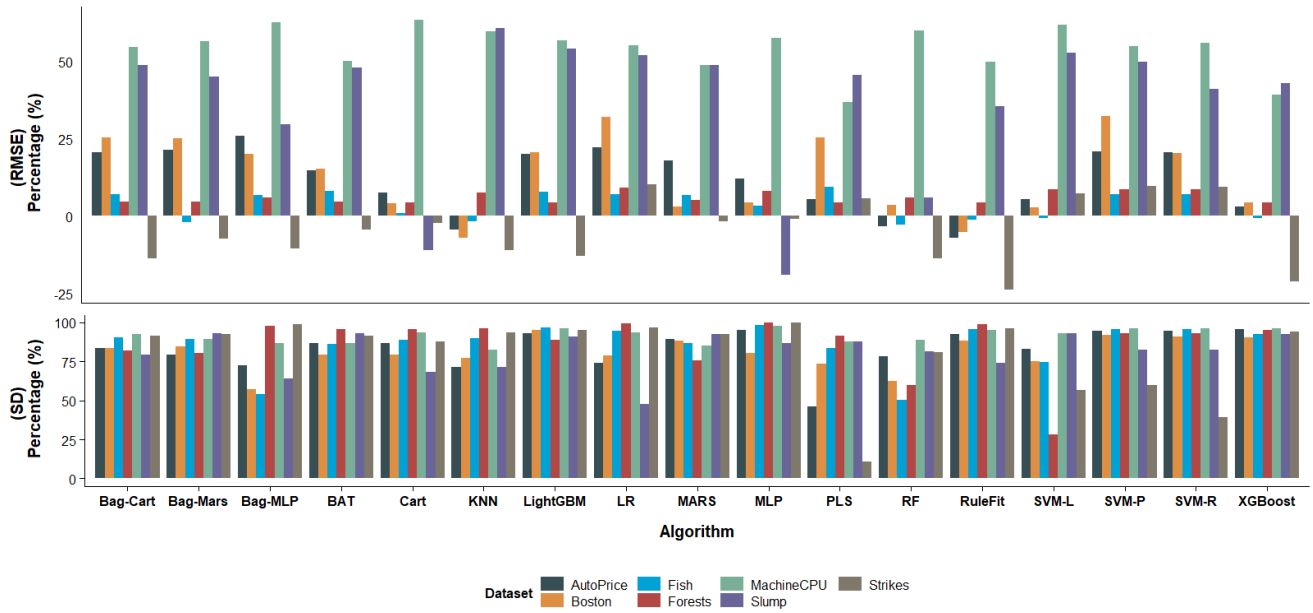



**FIGURE 4.** The training performance comparison between TP1-ELM and machine learning algorithms based on reduction rates.

benchmarked in the second step of the modeling process. Training and testing performances of TP1-ELM and TP2-ELM algorithms are summarized in Table 5, with training results of TP1-ELM visualized in Figure 4 and TP2-ELM in Figure 5. Additionally, reduction rates of test performances are displayed in Figure 6 for both proposed algorithms. Conclusions drawn from Table 5 and Figures 4-6 can be simplified as follows:

- In terms of training performance, both TP1-ELM and TP2-ELM algorithms achieved lower RMSE scores (up to 63.38% for TP1-ELM and 62.25% for TP2-ELM) compared to machine learning algorithms across most datasets. However, Rulefit, XGBoost, and Bagging (with Cart learner) performed better on the Strikes dataset. It appears that Rulefit and KNN algorithms are more competitive with the proposed algorithms compared to others.
- Regarding variability (standard deviation) of training performances, TP1-ELM and TP2-ELM algorithms exhibit lower deviation values (up to 99.75% for TP1-ELM and 99.78% for TP2-ELM) than all models across all datasets, indicating greater stability.
- Test performances (generalization performance) were found to be superior to machine learning models in most datasets, except for the Auto Price dataset (for some algorithms such as Random Forests, Bag-CART, etc.). Both proposed algorithms perform similarly, with TP1-ELM showing improvement of up to 56.79% and TP2-ELM achieving reduction of up to 51.11% in RMSE scores.
- In summary, the proposed algorithms offer more stable and generalizable results, with lower RMSE values, compared to both ELM-based and machine learning

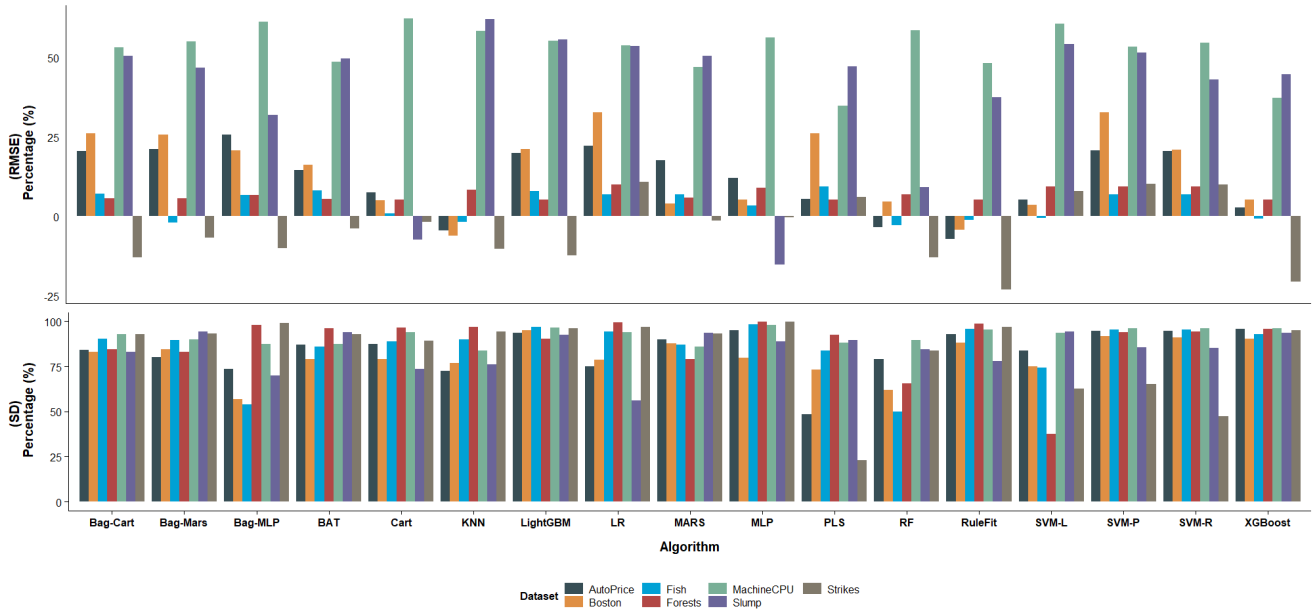



**FIGURE 5.** The training performance comparison between TP2-ELM and machine learning algorithms based on reduction rates.

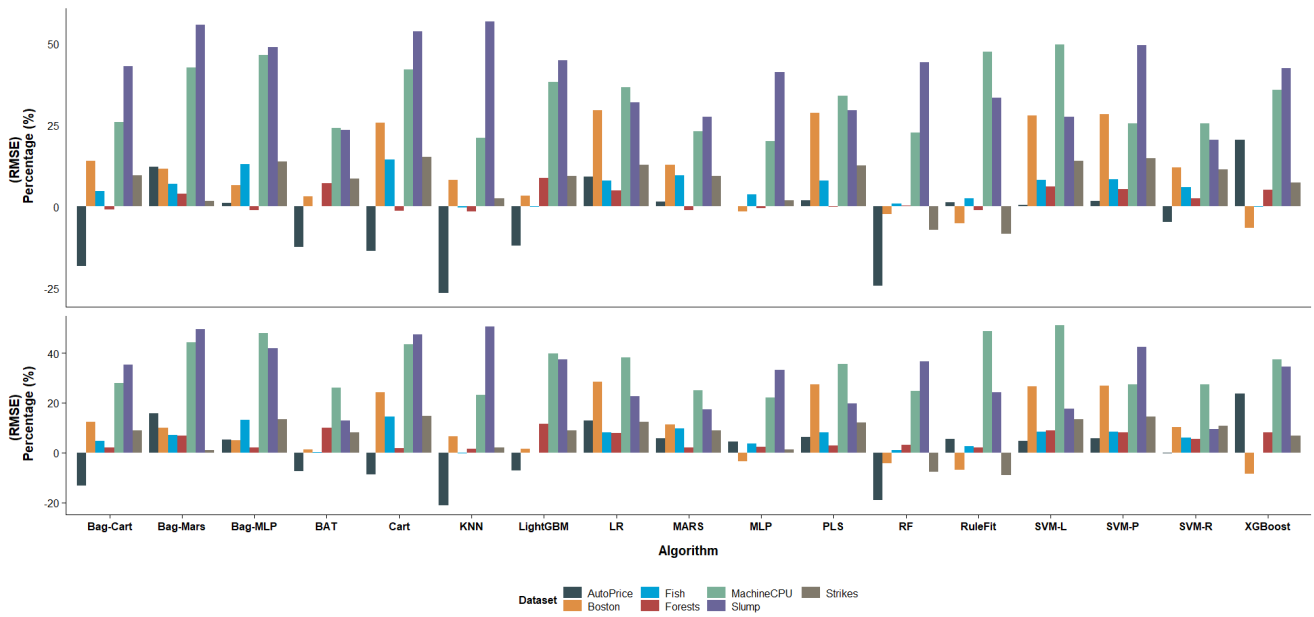



**FIGURE 6.** The testing performance comparison between the proposed models and machine learning algorithms based on reduction rates.

algorithms, especially for datasets with multicollinearity problems.

To facilitate more effective comparison of test performances for Slump and Boston datasets, the distribution of residual values calculated on the validation data is depicted in Figure 7 and Figure 8, respectively. The graphs clearly illustrate that the proposed algorithms exhibit more stable distributions of residual values around zero within narrower ranges compared to other algorithms. Therefore, it should be noted that the proposed algorithms can be considered as a robust alternative to ELM-based algorithms for all regression-oriented areas and problems, particularly in real-life applications dealing with multicollinearity. More specifically, it has important application areas in economic [71], [72], [73] and health sciences [72], [74] in which statistical estimators developed for linear models are successful.

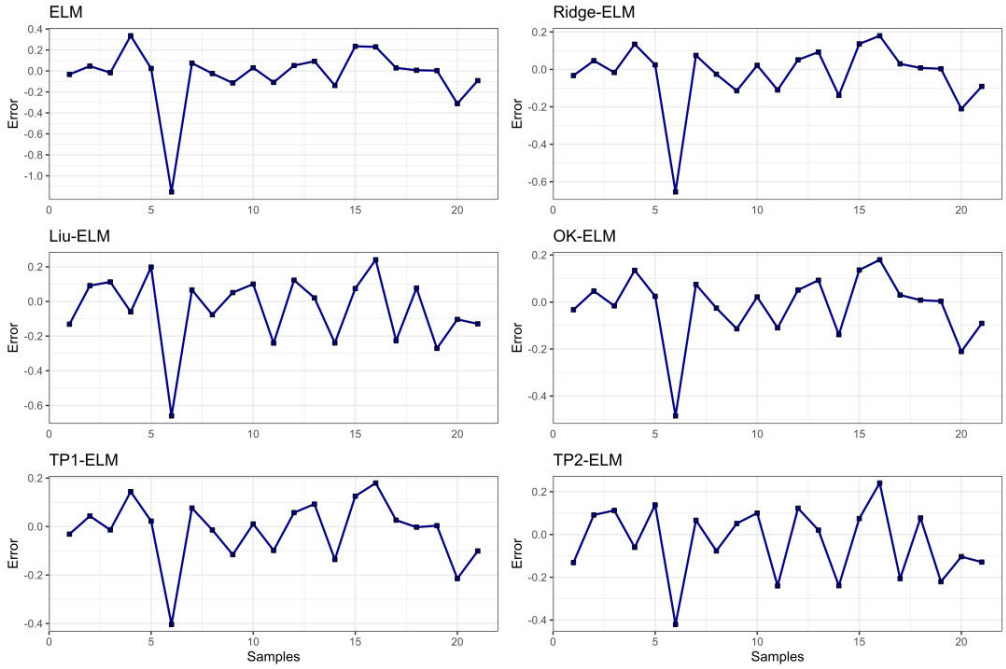



**FIGURE 7.** The testing error of the proposed algorithms based on slump testing data.

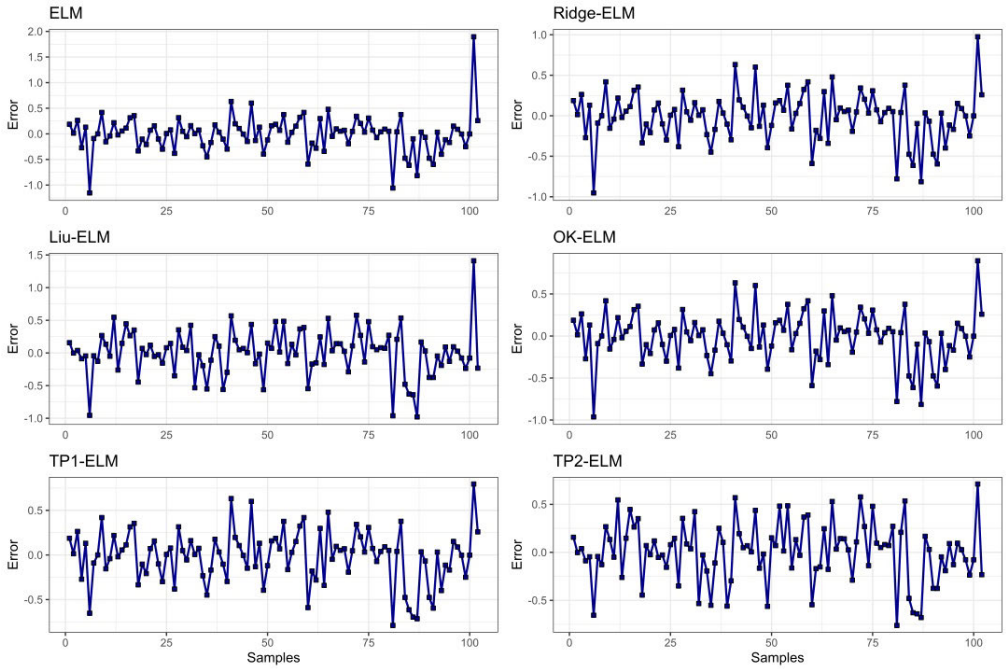



**FIGURE 8.** The testing error of the proposed algorithms based on boston testing data.

## V. CONCLUSION

In this paper, we introduce two novel statistical estimators (TP1-ELM and TP2-ELM) aimed at addressing the multicollinearity problem within the Extreme Learning Machine (ELM) algorithm. They stand out as an effective alternative to ridge regression (aka Tikhonov regularization), which is widely used in the field of machine learning, and are the foremost to be implemented in the context of feed-forward neural networks. Our proposed algorithms demonstrate superior performance over existing ELM-based and commonly used machine learning algorithms when applied to real-life datasets across

various domains, providing more generalizable and stable results.

It should also be noted that the proposed algorithms can be competitive with well-known machine learning algorithms on regular data apart from the data with multicollinearity problem, which is the focus of this study. Given their applicability to both classification and regression tasks in data-driven studies, these algorithms show promise as effective solutions to the multicollinearity problem.

Although our proposed algorithms exhibit satisfactory performance, a notable limitation of this study is the absence of various analytical methods from the literature for tuning parameter selection. Therefore, the development of analytical or heuristic methodologies for parameter estimation in algorithms involving statistical estimators is an open problem in the field of artificial neural networks (especially ELM). Additionally, future studies will focus on enhancing the proposed algorithms by incorporating feature selection capabilities to achieve more compact solutions, particularly in high-dimensional data settings where multicollinearity is prevalent. In conclusion, we plan to conduct a comprehensive comparison of the proposed algorithms on classification tasks as part of our future research goals by incorporating the limitations and identified gaps.

## REFERENCES

[1] S. Ding, H. Zhao, Y. Zhang, X. Xu, and R. Nie, "Extreme learning machine: Algorithm, theory and applications," *Artif. Intell. Rev.*, vol. 44, no. 1, pp. 103–115, Jun. 2015.

[2] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Netw.*, vol. 61, pp. 85–117, Jan. 2015.

[3] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, "Extreme learning machine: A new learning scheme of feedforward neural networks," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, Jul. 2004, pp. 985–990.

[4] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, "Extreme learning machine: Theory and applications," *Neurocomputing*, vol. 70, nos. 1–3, pp. 489–501, Dec. 2006.

[5] H. Yildirim and M. Revan Özkale, "The performance of ELM based ridge regression via the regularization parameters," *Expert Syst. Appl.*, vol. 134, pp. 225–233, Nov. 2019.

[6] L. Ma, W. Xie, Y. Zhang, and X. Feng, "Extreme learning machine based defect detection for solder joints," *J. Internet Technol.*, vol. 21, no. 5, pp. 1535–1543, 2020.

[7] P. Kansal, A. Kumar, and M. Gangadharappa, "Optimized extreme learning machine for intelligent spectrum sensing in 5G systems," *J. Commun. Technol. Electron.*, vol. 66, no. 3, pp. 322–332, Mar. 2021.

[8] M. K. Giri and S. Majumder, "Cooperative spectrum sensing using extreme learning machines for cognitive radio networks," *IETE Tech. Rev.*, vol. 39, no. 3, pp. 698–712, May 2022.

[9] G. Aldehim, M. A. Arasi, M. Khalid, S. S. Aljameel, R. Marzouk, H. Mohsen, I. Yaseen, and S. S. Ibrahim, "Gauss-mapping black widow optimization with deep extreme learning machine for Android malware classification model," *IEEE Access*, vol. 11, pp. 87062–87070, 2023.

[10] J. V. Kumar and S. M. Shaby, "Design and optimization of triangular microstrip patch antenna using extreme learning machine (ELM)-based improved crystal structure algorithm (CryStAl) for C-band application," *Int. J. Commun. Syst.*, vol. 37, no. 7, May 2024, Art. no. e5721.

[11] Z.-F. Liu, L.-L. Li, M.-L. Tseng, and M. K. Lim, "Prediction short-term photovoltaic power using improved chicken swarm optimizer–Extreme learning machine model," *J. Cleaner Prod.*, vol. 248, Mar. 2020, Art. no. 119272.

[12] W. Sun and C. Huang, "Predictions of carbon emission intensity based on factor analysis and an improved extreme learning machine from the perspective of carbon emission efficiency," *J. Cleaner Prod.*, vol. 338, Mar. 2022, Art. no. 130414.

[13] M. Ehteram, F. B. Banadkooki, and M. A. Nia, "Gaussian mutation-alpine skiing optimization algorithm-recurrent attention unit-gated recurrent unit-extreme learning machine model: An advanced predictive model for predicting evaporation," *Stochastic Environ. Res. Risk Assessment*, vol. 38, no. 5, pp. 1803–1830, May 2024.

[14] T. Hussain, S. M. Siniscalchi, H. S. Wang, Y. Tsao, V. M. Salerno, and W.-H. Liao, "Ensemble hierarchical extreme learning machine for speech dereverberation," *IEEE Trans. Cogn. Develop. Syst.*, vol. 12, no. 4, pp. 744–758, Dec. 2020.

[15] A. Miao and F. Liu, "Application of human motion recognition technology in extreme learning machine," *Int. J. Adv. Robot. Syst.*, vol. 18, no. 1, Jan. 2021, Art. no. 172988142098321.

[16] Z. Zhou, J. Ji, Y. Wang, Z. Zhu, and J. Chen, "Hybrid regression model via multivariate adaptive regression spline and online sequential extreme learning machine and its application in vision servo system," *Int. J. Adv. Robot. Syst.*, vol. 19, no. 3, May 2022, Art. no. 172988062211086.

[17] Z. Jin, G. Zhou, D. Gao, and Y. Zhang, "EEG classification using sparse Bayesian extreme learning machine for brain–computer interface," *Neural Comput. Appl.*, vol. 32, no. 11, pp. 6601–6609, Jun. 2020.

[18] M. Jiao, D. Wang, Y. Yang, and F. Liu, "More intelligent and robust estimation of battery state-of-charge with an improved regularized extreme learning machine," *Eng. Appl. Artif. Intell.*, vol. 104, Sep. 2021, Art. no. 104407.

[19] L. Sun, T. Wang, W. Ding, and J. Xu, "Partial multilabel learning using fuzzy neighborhood-based ball clustering and kernel extreme learning machine," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 7, pp. 2277–2291, Jul. 2023.

[20] S. Govindaraj, L. Raja, S. Velmurugan, and K. Vijayalakshmi, "Extreme learning machine optimized by artificial cell swarm optimization for the data fusion modal in WSNs," *Peer-to-Peer Netw. Appl.*, vol. 17, no. 3, pp. 1344–1357, May 2024.

[21] Z. Wang, S. Huo, X. Xiong, K. Wang, and B. Liu, "A maximally split and adaptive relaxed alternating direction method of multipliers for regularized extreme learning machines," *Mathematics*, vol. 11, no. 14, p. 3198, Jul. 2023.

[22] L. Wu, G. Huang, J. Fan, X. Ma, H. Zhou, and W. Zeng, "Hybrid extreme learning machine with meta-heuristic algorithms for monthly pan evaporation prediction," *Comput. Electron. Agricult.*, vol. 168, Jan. 2020, Art. no. 105115.

[23] M. Dogan, Y. S. Taspinar, I. Cinar, R. Kursun, I. A. Ozkan, and M. Koklu, "Dry bean cultivars classification using deep CNN features and salp swarm algorithm based extreme learning machine," *Comput. Electron. Agricult.*, vol. 204, Jan. 2023, Art. no. 107575.

[24] P. V. de Campos Souza and L. C. B. Torres, "Extreme wavelet fast learning machine for evaluation of the default profile on financial transactions," *Comput. Econ.*, vol. 57, no. 4, pp. 1263–1285, Apr. 2021.

[25] C. Zhang, A. Hu, and Y. Tian, "Daily tourism forecasting through a novel method based on principal component analysis, grey wolf optimizer, and extreme learning machine," *J. Forecasting*, vol. 42, no. 8, pp. 2121–2138, Dec. 2023.

[26] F. Özyurt, E. Sert, and D. Avcı, "An expert system for brain tumor detection: Fuzzy C-means with super resolution and convolutional neural network with extreme learning machine," *Med. Hypotheses*, vol. 134, Jan. 2020, Art. no. 109433.

[27] S. Govindarajan and R. Swaminathan, "Extreme learning machine based differentiation of pulmonary tuberculosis in chest radiographs using integrated local feature descriptors," *Comput. Methods Programs Biomed.*, vol. 204, Jun. 2021, Art. no. 106058.

[28] U. Ayman, M. S. Zia, O. D. Okon, N.-U. Rehman, T. Meraj, A. E. Ragab, and H. T. Rauf, "Epileptic patient activity recognition system using extreme learning machine method," *Biomedicines*, vol. 11, no. 3, p. 816, Mar. 2023.

[29] J. Wang, G. Athanasopoulos, R. J. Hyndman, and S. Wang, "Crude oil price forecasting based on internet concern using an extreme learning machine," *Int. J. Forecasting*, vol. 34, no. 4, pp. 665–677, Oct. 2018.

[30] S. Melina, H. Napitupulu, and N. Mohamed, "A conceptual model of investment-risk prediction in the stock market using extreme value theory with machine learning: A semisystematic literature review," *Risks*, vol. 11, no. 3, p. 60, Mar. 2023.

[31] F. Peng, C. Chen, D. Lv, N. Zhang, X. Wang, X. Zhang, and Z. Wang, "Gesture recognition by ensemble extreme learning machine based on surface electromyography signals," *Frontiers Hum. Neurosci.*, vol. 16, Jun. 2022, Art. no. 911204.

[32] Y. Ma, L. Wu, Y. Guan, and Z. Peng, "The capacity estimation and cycle life prediction of lithium-ion batteries using a new broad extreme learning machine approach," *J. Power Sources*, vol. 476, Nov. 2020, Art. no. 228581.

[33] Z. Chen, H. Yu, L. Luo, L. Wu, Q. Zheng, Z. Wu, S. Cheng, and P. Lin, "Rapid and accurate modeling of PV modules based on extreme learning machine and large datasets of I–V curves," *Appl. Energy*, vol. 292, Jun. 2021, Art. no. 116929.

[34] W. Dang, J. Guo, M. Liu, S. Liu, B. Yang, L. Yin, and W. Zheng, "A semi-supervised extreme learning machine algorithm based on the new weighted kernel for machine smell," *Appl. Sci.*, vol. 12, no. 18, p. 9213, Sep. 2022.

[35] N. M. Eldabah, A. Shoukry, W. Khair-Eldeen, S. Kobayashi, and M. A.-H. Gepreel, "Design and characterization of low Young's modulus Ti-Zr-Nb-based medium entropy alloys assisted by extreme learning machine for biomedical applications," *J. Alloys Compounds*, vol. 968, Dec. 2023, Art. no. 171755.

[36] V. Vapnik, *The Nature of Statistical Learning Theory*. New York, NY, USA: Springer, 2013.

[37] W. Deng, Q. Zheng, and L. Chen, "Regularized extreme learning machine," in *Proc. IEEE Symp. Comput. Intell. Data Mining*, Mar. 2009, pp. 389–395.

[38] G. Feng, G.-B. Huang, Q. Lin, and R. Gay, "Error minimized extreme learning machine with growth of hidden nodes and incremental learning," *IEEE Trans. Neural Netw.*, vol. 20, no. 8, pp. 1352–1357, Aug. 2009.

[39] Y. Yuan, Y. Wang, and F. Cao, "Optimization approximation solution for regression problem based on extreme learning machine," *Neurocomputing*, vol. 74, no. 16, pp. 2475–2482, Sep. 2011.

[40] S. Lu, X. Wang, G. Zhang, and X. Zhou, "Effective algorithms of the Moore–Penrose inverse matrices for extreme learning machine," *Intell. Data Anal.*, vol. 19, no. 4, pp. 743–760, Jul. 2015.

[41] A. E. Hoerl and R. W. Kennard, "Ridge regression: Applications to nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 69–82, Feb. 1970.

[42] L. Kejian, "A new class of blased estimate in linear regression," *Commun. Statist., Theory Methods*, vol. 22, no. 2, pp. 393–402, Jan. 1993.

[43] G.-B. Huang, D. H. Wang, and Y. Lan, "Extreme learning machines: A survey," *Int. J. Mach. Learn. Cybern.*, vol. 2, no. 2, pp. 107–122, Jun. 2011.

[44] G.-B. Huang, H. Zhou, X. Ding, and R. Zhang, "Extreme learning machine for regression and multiclass classification," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 42, no. 2, pp. 513–529, Apr. 2012.

[45] Y. Miche, A. Sorjamaa, P. Bas, O. Simula, C. Jutten, and A. Lendasse, "OP-ELM: Optimally pruned extreme learning machine," *IEEE Trans. Neural Netw.*, vol. 21, no. 1, pp. 158–162, Jan. 2010.

[46] Y. Miche, M. van Heeswijk, P. Bas, O. Simula, and A. Lendasse, "TROP-ELM: A double-regularized ELM using LARS and Tikhonov regularization," *Neurocomputing*, vol. 74, no. 16, pp. 2413–2421, Sep. 2011.

[47] J. M. Martínez-Martínez, P. Escandell-Montero, E. Soria-Olivas, J. D. Martín-Guerrero, R. Magdalena-Benedito, and J. Gómez-Sanchis, "Regularized extreme learning machine for regression problems," *Neurocomputing*, vol. 74, no. 17, pp. 3716–3721, Oct. 2011.

[48] G. Li and P. Niu, "An enhanced extreme learning machine based on ridge regression for regression," *Neural Comput. Appl.*, vol. 22, nos. 3–4, pp. 803–810, Mar. 2013.

[49] M. W. Fakhr, E. S. Youssef, and M. S. El-Mahallawy, "L1-regularized least squares sparse extreme learning machine for classification," in *Proc. Int. Conf. Inf. Commun. Technol. Res. (ICTRC)*, May 2015, pp. 222–225.

[50] X. Luo, X. Chang, and X. Ban, "Regression and classification using extreme learning machine based on $L_1$-norm and $L_2$-norm," *Neurocomputing*, vol. 174, pp. 179–186, Jan. 2016.

[51] B. He, T. Sun, T. Yan, Y. Shen, and R. Nian, "A pruning ensemble model of extreme learning machine with $L_{1/2}$ regularizer," *Multidimensional Syst. Signal Process.*, vol. 28, pp. 1051–1069, Jul. 2017.

[52] P. Shan, Y. Zhao, X. Sha, Q. Wang, X. Lv, S. Peng, and Y. Ying, "Interval LASSO regression based extreme learning machine for nonlinear multivariate calibration of near infrared spectroscopic datasets," *Anal. Methods*, vol. 10, no. 25, pp. 3011–3022, 2018.

[53] H. Yıldırım and M. R. Özkale, "An enhanced extreme learning machine based on liu regression," *Neural Process. Lett.*, vol. 52, no. 1, pp. 421–442, Aug. 2020.

[54] H. Yıldırım and M. R. Özkale, "LL-ELM: A regularized extreme learning machine based on $L_1$-norm and Liu estimator," *Neural Comput. Appl.*, vol. 33, no. 16, pp. 10469–10484, Aug. 2021.

[55] J. Li and G. Zhao, "Tensor-based type-2 extreme learning machine with $L_1$-norm and Liu regression," in *Proc. 41st Chin. Control Conf. (CCC)*, Jul. 2022, pp. 6327–6333.

[56] M. R. Özkale and S. Kaçiranlar, "The restricted and unrestricted two-parameter estimators," *Commun. Statist., Theory Methods*, vol. 36, no. 15, pp. 2707–2725, Nov. 2007.

[57] H. Yıldırım and M. R. Özkale, "A combination of ridge and Liu regressions for extreme learning machine," *Soft Comput.*, vol. 27, no. 5, pp. 2493–2508, Mar. 2023.

[58] S. Sakallıoğlu and S. Kaçıranlar, "A new biased estimator based on ridge estimation," *Stat. Papers*, vol. 49, no. 4, pp. 669–689, Oct. 2008.

[59] H. Yang and X. Chang, "A new two-parameter estimator in linear regression," *Commun. Statist., Theory Methods*, vol. 39, no. 6, pp. 923–934, Mar. 2010.

[60] C. R. Rao and M. B. Rao, *Matrix Algebra and Its Applications to Statistics and Econometrics*. Singapore: World Scientific, 1998.

[61] J. R. Schott, *Matrix Analysis for Statistics*. Hoboken, NJ, USA: Wiley, 2016.

[62] G. Tutz and H. Binder, "Boosting ridge regression," *Comput. Statist. Data Anal.*, vol. 51, no. 12, pp. 6044–6059, Aug. 2007.

[63] A. Asuncion and D. Newman. (2007). *UCI Machine Learning Repository*. [Online]. Available: https://archive.ics.uci.edu

[64] Kaggle. (2024). *Datasets*. [Online]. Available: https://www.kaggle.com/datasets

[65] L. Torgo. (2024). *Datasets*. [Online]. Available: https://www.dcc.fc.up.pt/~ltorgo/Regression/price.html

[66] Carnegie Mellon University. (2024). *Datasets*. [Online]. Available: https://lib.stat.cmu.edu/datasets/strikes

[67] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*. Hoboken, NJ, USA: Wiley, 2021.

[68] A. Ali, M. Hamraz, P. Kumam, D. M. Khan, U. Khalil, M. Sulaiman, and Z. Khan, "A k-nearest neighbours based ensemble via optimal model selection for regression," *IEEE Access*, vol. 8, pp. 132095–132105, 2020.

[69] A. Ali, M. Hamraz, N. Gul, D. M. Khan, S. Aldahmani, and Z. Khan, "A k nearest neighbour ensemble via extended neighbourhood rule and feature subsets," *Pattern Recognit.*, vol. 142, Oct. 2023, Art. no. 109641.

[70] A. Ali, Z. Khan, D. M. Khan, and S. Aldahmani, "An optimal random projection k nearest neighbors ensemble via extended neighborhood rule for binary classification," *IEEE Access*, vol. 12, pp. 61401–61409, 2024.

[71] K. C. Arum, F. I. Ugwuowo, H. E. Oranye, T. O. Alakija, T. E. Ugah, and O. C. Asogwa, "Combating outliers and multicollinearity in linear regression model using robust Kibria–Lukman mixed with principal component estimator, simulation and computation," *Sci. Afr.*, vol. 19, Mar. 2023, Art. no. e01566.

[72] I. S. Dar, S. Chand, M. Shabbir, and B. M. G. Kibria, "Condition-index based new ridge regression estimator for linear regression model with multicollinearity," *Kuwait J. Sci.*, vol. 50, no. 2, pp. 91–96, Apr. 2023.

[73] S. Yasin, S. Kamal, and M. Suhail, "Performance of some new ridge parameters in two-parameter ridge regression model," *Iranian J. Sci. Technol., Trans. A, Sci.*, vol. 45, no. 1, pp. 327–341, Feb. 2021.

[74] A. F. Lukman, B. M. G. Kibria, C. K. Nziku, M. Amin, E. T. Adewuyi, and R. Farghali, "K-L estimator: Dealing with multicollinearity in the logistic regression model," *Mathematics*, vol. 11, no. 2, p. 340, Jan. 2023.

**HASAN YILDIRIM** received the Bachelor of Science degree from Anadolu University and the master's and Ph.D. degrees in statistics from Çukurova University, Türkiye. Since 2012, he has been a Faculty Member with Karamanoğlu Mehmetbey University, where he is currently an Assistant Professor Doctor. His research interests include statistical learning, artificial intelligence, challenges associated with high-dimensional data, and biostatistics.

# Regularized Extreme Learning Machine (ELM) Addressing Multicollinearity

ML Assignment, Allen Sumanth A (108123012) & Adithya S (108123006)

## Data Pipeline & VIF Diagnostics

We begin by establishing a data pipeline that fetches the 7 tabular regression datasets specified in the paper: **Auto Price, Boston Housing, Fish, Forests, Machine CPU, Servo, and Slump/Strikes**. We then define a Variance Inflation Factor (VIF) diagnostic function to demonstrate the presence of multicollinearity to find out which datasets have this problem.

In [3]:

```
import charset_normalizer
import charset_normalizer
import numpy as np
import pandas as pd
np.asnumpy = np.asarray
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

def fetch_paper_datasets():
    import pandas as pd
    from sklearn.datasets import fetch_openml

    # Bypass OpenML for Fish and Servo for proper datasets
    configs = {
        'Auto Price': {'id': 195},
        'Boston Housing': {'id': 531},
        'Fish': {'url': 'https://archive.ics.uci.edu/ml/machine-learning-databases/00504/qsar_fish_toxicity.csv', 'sep': ';', 'header': None},
        'Forests': {'name': 'forest_fires', 'version': 1},
        'Machine CPU': {'id': 199},
        'Servo': {'url': 'https://archive.ics.uci.edu/ml/machine-learning-databases/servo/servo.data', 'sep': ',', 'header': None},
        'Slump': {'name': 'slump'},
        'Strikes': {'name': 'strikes'}
    }

    datasets = {}
    for name, cfg in configs.items():
        try:
            # print(f"Fetching {name}...")
            if 'url' in cfg:
                # Direct URI retrieval for the other cases
                df = pd.read_csv(cfg['url'], sep=cfg['sep'], header=cfg['header'])
                # Target is always localized to the last column by UCI standard
                X = df.iloc[:, :-1]
                y = df.iloc[:, -1]
            else:
                # Normal OpenML case
                if 'id' in cfg:
                    X, y = fetch_openml(data_id=cfg['id'], as_frame=True, return_X_y=True, parser='auto')
                else:
                    version = cfg.get('version', 'active')
                    if version == 'active':
                        X, y = fetch_openml(name=cfg['name'], as_frame=True, return_X_y=True, parser='auto')
                    else:
                        X, y = fetch_openml(name=cfg['name'], version=version, as_frame=True, return_X_y=True, parser='auto')

                df = pd.concat([X, y], axis=1)
                target_col = df.columns[-1]
                df = df.dropna(subset=[target_col])
                X = df.iloc[:, :-1]
                y = df.iloc[:, -1]

            # Guarantee continuous float targeting
            y = pd.to_numeric(y, errors='coerce')

            # Remove NaN entries
            valid_idx = y.notna()
            X = X[valid_idx].dropna()
            y = y[valid_idx].loc[X.index]

            # Categorical encoding - converting types into one hot vectors
            X = pd.get_dummies(X, drop_first=True, dtype=float)

            datasets[name] = {'X': X, 'y': y.values.reshape(-1, 1)}
            # To check the input vector size and shape
```

```
            print(f"Loaded {name}: X shape {X.shape}")

        except Exception as e:
            print(f"Failed to fetch {name}: {e}")

    return datasets
# VIF function - uses correlation matrix inverse
def calculate_vif(X_df):
    corr_matrix = X_df.corr().values
    inv_corr = np.linalg.inv(corr_matrix + np.eye(corr_matrix.shape[0]) * 1e-8)
    vif_values = np.diag(inv_corr)

    vif_df = pd.DataFrame({
        'Feature': X_df.columns,
        'VIF': vif_values
    })
    return vif_df.sort_values(by='VIF', ascending=False)

# testing w ex dataset
if __name__ == '__main__':
    datasets = fetch_paper_datasets()

    if 'Machine CPU' in datasets:
        print("\n*** VIF Diagnostics for Machine CPU ***")
        X_cpu = datasets['Machine CPU']['X']
        vif_df = calculate_vif(X_cpu)
        print(vif_df.head())

        print("\n*** Data Split & Scale ***")
        y_cpu = datasets['Machine CPU']['y']

        X_train, X_test, y_train, y_test = train_test_split(X_cpu.values, y_cpu, test_size=0.2, random_state=42)

        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        print(f"X_train_scaled: {X_train_scaled.shape}, X_test_scaled: {X_test_scaled.shape}")
```

```
Loaded Auto Price: X shape (159, 19)
Loaded Boston Housing: X shape (506, 20)
Loaded Fish: X shape (908, 6)
Loaded Forests: X shape (517, 27)
Loaded Machine CPU: X shape (125, 6)
Loaded Servo: X shape (167, 10)
Loaded Slump: X shape (103, 9)
Loaded Strikes: X shape (625, 6)

*** VIF Diagnostics for Machine CPU ***
      Feature           VIF
2  PARTNERS_1  3.750000e+07
3  PARTNERS_8  3.750000e+07
5      TYPE_9  2.500000e+07
0      THORAX  2.655853e+00
1       SLEEP  2.022799e+00

*** Data Split & Scale ***
X_train_scaled: (100, 6), X_test_scaled: (25, 6)
```

## Define All ELM Classes

Here we define the standard `BaseELM` and all regularized counterparts: `RidgeELM`, `LiuELM`, `OK_ELM`, `TP1_ELM`, and `TP2_ELM`. Crucially, for regularized classes where the diagonal is modified ($H^T H + kI_m$), we use direct CPU algebraic solvers (like `scipy.linalg.solve` or native `numpy.linalg.inv`) instead of computationally expensive SVD pseudoinverse approximations. This works because the $kI_m$ part is guaranteed to make the term invertible, so no need for Moore-Penrose Inverse or other techniques.

In [4]:
```
import numpy as np

class BaseELM:
    def __init__(self, n_hidden=100, random_state=None):
        self.n_hidden = n_hidden
        self.random_state = random_state
        self.input_weights = None
        self.biases = None
        self.beta = None

    def _hidden_output(self, X):
        X = np.asarray(X)
        if self.input_weights is None:
            if self.random_state is not None:
                np.random.seed(self.random_state)
            n_features = X.shape[1]
```

```python
            # Standard uniform weights
            self.input_weights = np.random.uniform(-1, 1, (n_features, self.n_hidden))
            self.biases = np.random.uniform(-1, 1, (self.n_hidden,))

        # Sigmoid or Sine activation; we'll use sine for broad applicability as per standard ELM benchmark
        H = np.sin(np.dot(X, self.input_weights) + self.biases)
        return H

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)

        # Base ELM requires pinv since no regularization is guaranteed
        self.beta = np.linalg.pinv(H) @ y
        return self

    def predict(self, X):
        X = np.asarray(X)
        H = self._hidden_output(X)
        y_pred = np.dot(H, self.beta)
        return np.asnumpy(y_pred) # Convert back to standard array

class RidgeELM(BaseELM):
    def __init__(self, k, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # SMART MATH HACK: HT_H + kI is invertible. Use inv() instead of pinv().
        self.beta = np.linalg.inv(HT_H + self.k * I) @ (HT @ y)
        return self

class LiuELM(BaseELM):
    def __init__(self, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        beta_elm = np.linalg.pinv(H) @ y
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # Liu Estimator: (H^TH + I)^-1 (H^TY + d*beta_elm)
        # Using inv() since + I ensures invertibility
        self.beta = np.linalg.inv(HT_H + I) @ (HT @ y + self.d * beta_elm)
        return self

class OK_ELM(BaseELM):
    def __init__(self, k, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
```

```
        beta_elm = np.linalg.pinv(H) @ y
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # OK-ELM: convex comb. -> (H^TH + kI)^-1 (H^TY + kd*beta_elm)
        self.beta = np.linalg.inv(HT_H + self.k * I) @ (HT @ y + self.k * self.d * beta_elm)
        return self

class TP1_ELM(BaseELM):
    def __init__(self, k, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # Compute beta_ridge first
        inv_HT_H_kI = np.linalg.inv(HT_H + self.k * I)
        beta_ridge = inv_HT_H_kI @ (HT @ y)

        # TP1-ELM: (H^TH + kI)^-1 (H^TY + d*beta_ridge)
        self.beta = inv_HT_H_kI @ (HT @ y + self.d * beta_ridge)
        return self

class TP2_ELM(BaseELM):
    def __init__(self, k, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # TP2-ELM: (H^TH + I)^-1 (H^TH + dI) (H^TH + kI)^-1 H^TY
        term1 = np.linalg.inv(HT_H + I)
        term2 = HT_H + self.d * I
        term3 = np.linalg.inv(HT_H + self.k * I)

        self.beta = term1 @ term2 @ term3 @ (HT @ y)
        return self

print("BaseELM and Regularized ELM variants (Ridge, Liu, OK, TP1, TP2) successfully built in NumPy.")
```

```
BaseELM and Regularized ELM variants (Ridge, Liu, OK, TP1, TP2) successfully built in NumPy.
```

## Define Traditional ML Baselines

We initialize the standard Machine Learning models for performance benchmarking:

1. **Ridge Regression** ( `sklearn.linear_model.Ridge` )
2. **Random Forest** ( `sklearn.ensemble.RandomForestRegressor` )
3. **XGBoost** ( `xgboost.XGBRegressor` )

We also define parameter grids compatible with the randomized search mechanism we'll deploy in the final experimental loop.

In [5]:
```
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
import numpy as np

# Instantiate comparison ML models (ELM ones)
traditional_models = {
```

```
    'Ridge': Ridge(),
    'RandomForest': RandomForestRegressor(random_state=42, n_jobs=-1),
    'XGBoost': XGBRegressor(random_state=42, n_jobs=-1, eval_metric='rmse')
}

# Grids for RandomizedSearchCV sampling during the 5-fold CV phase
traditional_grids = {
    'Ridge': {
        'alpha': np.logspace(-5, 1, 50)
    },
    'RandomForest': {
        'n_estimators': [50, 100, 200, 300],
        'max_depth': [None, 5, 10, 20],
        'min_samples_split': [2, 5, 10]
    },
    'XGBoost': {
        'n_estimators': [50, 100, 200],
        'learning_rate': [0.01, 0.05, 0.1, 0.2],
        'max_depth': [3, 5, 7, 9],
        'subsample': [0.8, 1.0]
    }
}

print("Traditional ML models (Ridge, RandomForest, XGBoost) and parameter grids initialized.")
```

```
Traditional ML models (Ridge, RandomForest, XGBoost) and parameter grids initialized.
```

## Prototype Verification

Before running the massive experimental loop over all datasets, we will execute a localized test on the **Auto Price** dataset. This demonstrates the execution speed advantage of extreme learning parameters computed via deterministic matrix operations natively on NumPy (CPU) over standard gradient-based trees (XGBoost). We evaluate `BaseELM`, `TP1_ELM`, and `XGBRegressor` side-by-side.

In [6]:

```
import time
from sklearn.metrics import mean_squared_error

if 'datasets' not in locals() and 'datasets' not in globals():
    datasets = fetch_paper_datasets()

if 'Auto Price' in datasets:
    print("--- Prototype Verification: CPU ELM vs XGBoost on 'Auto Price' ---")
    X_auto = datasets['Auto Price']['X'].values
    y_auto = datasets['Auto Price']['y']

    # Standard splitting and scaling
    X_train_proto, X_test_proto, y_train_proto, y_test_proto = train_test_split(X_auto, y_auto, test_size=0.2, random_state=42)
    scaler_proto = StandardScaler()
    X_train_proto_s = scaler_proto.fit_transform(X_train_proto)
    X_test_proto_s = scaler_proto.transform(X_test_proto)

    # BaseELM (Using pinv)
    start = time.time()
    base_elm = BaseELM(n_hidden=100, random_state=42)
    base_elm.fit(X_train_proto_s, y_train_proto)
    base_preds = base_elm.predict(X_test_proto_s)
    base_time = time.time() - start
    base_rmse = np.sqrt(mean_squared_error(y_test_proto, base_preds))
    print(f"BaseELM       | RMSE: {base_rmse:10.4f} | Time: {base_time:.4f}s")

    # TP1_ELM (Using Smart Math inv)
    start = time.time()
    # arbitrary regularization values for verification
    tp1_elm = TP1_ELM(k=0.1, d=0.1, n_hidden=100, random_state=42)
    tp1_elm.fit(X_train_proto_s, y_train_proto)
    tp1_preds = tp1_elm.predict(X_test_proto_s)
    tp1_time = time.time() - start
    tp1_rmse = np.sqrt(mean_squared_error(y_test_proto, tp1_preds))
    print(f"TP1_ELM (CPU) | RMSE: {tp1_rmse:10.4f} | Time: {tp1_time:.4f}s")

    # XGBRegressor
    start = time.time()
    xgb_proto = traditional_models['XGBoost']
    # flatten y for XGBoost
    xgb_proto.fit(X_train_proto_s, y_train_proto.ravel())
    xgb_preds = xgb_proto.predict(X_test_proto_s)
    xgb_time = time.time() - start
    xgb_rmse = np.sqrt(mean_squared_error(y_test_proto, xgb_preds))
    print(f"XGBRegressor  | RMSE: {xgb_rmse:10.4f} | Time: {xgb_time:.4f}s")
else:
    print("Err.")
```

```
--- Prototype Verification: CPU ELM vs XGBoost on 'Auto Price' ---
BaseELM       | RMSE: 21819.8309 | Time: 0.0778s
TP1_ELM (CPU) | RMSE: 17826.5632 | Time: 0.0142s
XGBRegressor  | RMSE:  1938.2692 | Time: 1.1097s
```

## Main Experiment

This corresponds to the benchmarking phase in the paper over the 7 datasets. For each dataset:

- We run **30 independent trials** (matching the paper).
- For each trial, we perform a standard **5-fold Cross Validation** over the full dataset.
- For regularization parameters ($k$ and $d$), we sample 15 random combinations from defined value ranges.
- We track both **train RMSE** and **test (CV-fold) RMSE**, plus the **coefficient norm** per model.
- We report **Mean ± Std** of RMSE and norm over the 30 trials.
- After the loop, the **Reduction Rate (RR%)** of TP1-ELM and TP2-ELM against every baseline is computed.

In [7]:

```python
# import warnings
# warnings.filterwarnings("ignore")

from sklearn.model_selection import KFold, RandomizedSearchCV
from sklearn.model_selection import train_test_split as tts
from tqdm.auto import tqdm
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error


# Hyperparameter Spaces (Not using the whole range given in the paper)
k_space = [1e-5, 1e-4, 1e-3, 1e-2, 0.05, 0.1, 0.5, 1.0]
d_space  = [1e-5, 1e-4, 1e-3, 1e-2, 0.05, 0.1, 0.5, 1.0]

np.random.seed(42)
kd_combinations = list(zip(np.random.choice(k_space, 15), np.random.choice(d_space, 15)))

# tst mode for faster exec (suppressing warns)
TEST_MODE = False
n_trials = 1 if TEST_MODE else 30
n_splits = 2 if TEST_MODE else 5
n_iter_search  = 1 if TEST_MODE else 5
kd_combinations = kd_combinations[:2] if TEST_MODE else kd_combinations


results_dict = {}

for ds_name, ds_data in tqdm(datasets.items(), desc="Datasets Progress"):
    if ds_name in results_dict:
        continue

    X_full = ds_data['X'].values
    y_full = ds_data['y']

    if TEST_MODE:
        X_full = X_full[:50]
        y_full = y_full[:50]

    if len(X_full) < n_splits:
        print(f"Skipping {ds_name} - Insufficient valid rows ({len(X_full)}).")
        continue

    MODEL_NAMES = ['BaseELM', 'RidgeELM', 'LiuELM', 'OK_ELM', 'TP1_ELM', 'TP2_ELM',
                   'Ridge', 'RandomForest', 'XGBoost']

    trial_test_scores  = {m: [] for m in MODEL_NAMES}
    trial_train_scores = {m: [] for m in MODEL_NAMES}
    trial_norms        = {m: [] for m in MODEL_NAMES}

    for trial in tqdm(range(n_trials), leave=False, desc=f"{ds_name} Trials"):
        kf = KFold(n_splits=n_splits, shuffle=True, random_state=trial)
        fold_test  = {m: [] for m in MODEL_NAMES}
        fold_train = {m: [] for m in MODEL_NAMES}
        fold_norms = {m: [] for m in MODEL_NAMES}

        for fold_idx, (train_idx, test_idx) in enumerate(kf.split(X_full)):
            X_tr, X_te = X_full[train_idx], X_full[test_idx]
            y_tr, y_te = y_full[train_idx], y_full[test_idx]

            sc = StandardScaler()
```

```python
            X_tr_s = sc.fit_transform(X_tr)
            X_te_s = sc.transform(X_te)
            y_tr_1d = y_tr.ravel()

            X_tune_tr, X_tune_val, y_tune_tr, y_tune_val = tts(
                X_tr_s, y_tr, test_size=0.2, random_state=trial * n_splits + fold_idx
            )

            # Old school models
            for t_name, t_model in traditional_models.items():
                t_grid   = traditional_grids[t_name]
                r_search = RandomizedSearchCV(
                    t_model, param_distributions=t_grid,
                    n_iter=n_iter_search, cv=2 if TEST_MODE else 3,
                    n_jobs=-1, random_state=trial
                )
                with warnings.catch_warnings():
                    warnings.simplefilter("ignore")
                    r_search.fit(X_tr_s, y_tr_1d)
                best = r_search.best_estimator_
                fold_test[t_name].append(
                    np.sqrt(mean_squared_error(y_te, best.predict(X_te_s)))
                )
                fold_train[t_name].append(
                    np.sqrt(mean_squared_error(y_tr_1d, best.predict(X_tr_s)))
                )
                fold_norms[t_name].append(
                    np.linalg.norm(best.coef_) if t_name == 'Ridge' else 0.0
                )

            # BaseELM - no hyperparameters
            belm = BaseELM(n_hidden=100, random_state=trial).fit(X_tr_s, y_tr)
            fold_test['BaseELM'].append(
                np.sqrt(mean_squared_error(y_te, belm.predict(X_te_s)))
            )
            fold_train['BaseELM'].append(
                np.sqrt(mean_squared_error(y_tr, belm.predict(X_tr_s)))
            )
            b_beta = belm.beta
            if hasattr(b_beta, 'get'): b_beta = b_beta.get()
            fold_norms['BaseELM'].append(np.linalg.norm(b_beta))

            # Regularized ELMs
            best_tune_rmse = {m: float('inf')
                              for m in ['RidgeELM', 'LiuELM', 'OK_ELM', 'TP1_ELM', 'TP2_ELM']}
            best_k         = {m: None for m in best_tune_rmse}
            best_d         = {m: None for m in best_tune_rmse}

            for k, d in kd_combinations:
                # RidgeELM
                m = 'RidgeELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    RidgeELM(k=k, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m] = r, k

                # LiuELM
                m = 'LiuELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    LiuELM(d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_d[m] = r, d

                # OK_ELM
                m = 'OK_ELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    OK_ELM(k=k, d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m], best_d[m] = r, k, d

                # TP1_ELM
                m = 'TP1_ELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    TP1_ELM(k=k, d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
```

```python
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m], best_d[m] = r, k, d

                # TP2_ELM
                m = 'TP2_ELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    TP2_ELM(k=k, d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m], best_d[m] = r, k, d

            # retrain
            for m, Cls, kwargs in [
                ('RidgeELM', RidgeELM, dict(k=best_k['RidgeELM'])),
                ('LiuELM',  LiuELM,  dict(d=best_d['LiuELM'])),
                ('OK_ELM',  OK_ELM,  dict(k=best_k['OK_ELM'],  d=best_d['OK_ELM'])),
                ('TP1_ELM', TP1_ELM, dict(k=best_k['TP1_ELM'], d=best_d['TP1_ELM'])),
                ('TP2_ELM', TP2_ELM, dict(k=best_k['TP2_ELM'], d=best_d['TP2_ELM'])),
            ]:
                final_mdl = Cls(**kwargs, n_hidden=100, random_state=trial).fit(X_tr_s, y_tr)
                # True test RMSE — unseen fold
                fold_test[m].append(
                    np.sqrt(mean_squared_error(y_te, final_mdl.predict(X_te_s)))
                )
                # Train RMSE on full fold train set
                fold_train[m].append(
                    np.sqrt(mean_squared_error(y_tr, final_mdl.predict(X_tr_s)))
                )
                opt_beta = final_mdl.beta
                if hasattr(opt_beta, 'get'): opt_beta = opt_beta.get()
                fold_norms[m].append(np.linalg.norm(opt_beta))

        # Accumulate trial-level means
        for m in MODEL_NAMES:
            trial_test_scores[m].append(np.mean(fold_test[m]))
            trial_train_scores[m].append(np.mean(fold_train[m]))
            trial_norms[m].append(np.mean(fold_norms[m]))

    # Build summary table
    ds_results = {}
    for m in MODEL_NAMES:
        te = trial_test_scores[m]
        tr = trial_train_scores[m]
        no = trial_norms[m]
        ds_results[f"{m} | Train RMSE"] = f"{np.mean(tr):.4f} \u00b1 {np.std(tr):.4f}"
        ds_results[f"{m} | Test RMSE"]  = f"{np.mean(te):.4f} \u00b1 {np.std(te):.4f}"
        ds_results[f"{m} | Norm"]       = (
            f"{np.mean(no):.4f} \u00b1 {np.std(no):.4f}"
            if m not in ['RandomForest', 'XGBoost'] else "N/A"
        )

    results_dict[ds_name] = ds_results

# Reduction Rate (RR%)
from IPython.display import display, Markdown
display(Markdown("### Reduction Rate (RR%)"))

def get_mean(ds, model_key):
    val = results_dict[ds].get(f"{model_key} | Test RMSE", None)
    return float(val.split(" \u00b1 ")[0]) if val else None

rr_rows = []
baselines = ['BaseELM', 'RidgeELM', 'LiuELM', 'OK_ELM', 'Ridge', 'RandomForest', 'XGBoost']
for ds_name in results_dict:
    for tp in ['TP1_ELM', 'TP2_ELM']:
        tp_mean = get_mean(ds_name, tp)
        if tp_mean is None:
            continue
        row = {'Dataset': ds_name, 'Proposed': tp}
        for bl in baselines:
            bl_mean = get_mean(ds_name, bl)
            row[bl] = (
                f"{(bl_mean - tp_mean) / bl_mean * 100:.2f}%"
                if bl_mean else 'N/A'
            )
        rr_rows.append(row)

rr_df = pd.DataFrame(rr_rows).set_index(['Dataset', 'Proposed'])
from IPython.display import display, HTML
display(rr_df)
```

```python
# Full Results Table
final_df = pd.DataFrame(results_dict).T
display(Markdown("### Full Results (Norm = Mean ± Std)"))
final_df
```

```
Datasets Progress:   0%|          | 0/8 [00:00<?, ?it/s]
Auto Price Trials:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
/home/allen/Allen/Acads/6th sem/acads/ml/project-notebook/.venv/lib/python3.14/site-packages/sklearn/utils/parallel.py:144: UserWarning: `sklearn.utils.parallel.delayed` should be used with `sklearn.utils.parallel.Parallel` to make it possible to p
ropagate the scikit-learn configuration of the current thread to the joblib workers.
  warnings.warn(
/home/allen/Allen/Acads/6th sem/acads/ml/project-notebook/.venv/lib/python3.14/site-packages/sklearn/utils/parallel.py:144: UserWarning: `sklearn.utils.parallel.delayed` should be used with `sklearn.utils.parallel.Parallel` to make it possible to p
ropagate the scikit-learn configuration of the current thread to the joblib workers.
  warnings.warn(

                    ...warnings truncated for presentation
```

### Reduction Rate (RR%)

| | | BaseELM | RidgeELM | LiuELM | OK_ELM | Ridge | RandomForest | XGBoost |
|---|---|---|---|---|---|---|---|---|
| **Dataset** | **Proposed** | | | | | | | |
| **Auto Price** | **TP1_ELM** | 54.46% | -0.37% | -1.58% | 21.35% | -315.80% | -446.74% | -449.27% |
| | **TP2_ELM** | 58.61% | 8.76% | 7.67% | 28.51% | -277.96% | -396.98% | -399.28% |
| **Boston Housing** | **TP1_ELM** | 1.16% | -0.02% | -0.02% | -0.01% | -210.19% | -346.77% | -376.84% |
| | **TP2_ELM** | 2.16% | 0.99% | 0.99% | 1.00% | -207.06% | -342.25% | -372.02% |
| **Fish** | **TP1_ELM** | 3.68% | -0.22% | -0.28% | -0.25% | -28.64% | -36.04% | -34.87% |
| | **TP2_ELM** | 4.53% | 0.66% | 0.60% | 0.63% | -27.51% | -34.84% | -33.68% |
| **Forests** | **TP1_ELM** | 0.37% | -0.01% | -0.01% | -0.01% | -22.31% | -13.19% | -12.73% |
| | **TP2_ELM** | 0.75% | 0.37% | 0.37% | 0.37% | -21.84% | -12.76% | -12.30% |
| **Machine CPU** | **TP1_ELM** | 99.75% | -1.34% | 36.51% | 56.58% | -25.05% | -17.13% | -18.56% |
| | **TP2_ELM** | 99.76% | 4.57% | 40.21% | 59.11% | -17.76% | -10.30% | -11.65% |
| **Servo** | **TP1_ELM** | 39.99% | -0.09% | -0.27% | 0.18% | -24.53% | -68.25% | -94.88% |
| | **TP2_ELM** | 41.57% | 2.55% | 2.37% | 2.81% | -21.25% | -63.82% | -89.75% |
| **Slump** | **TP1_ELM** | 20.12% | -12.64% | -13.39% | -12.70% | -551.15% | -365.26% | -408.36% |
| | **TP2_ELM** | 30.75% | 2.35% | 1.70% | 2.30% | -464.49% | -303.35% | -340.71% |
| **Strikes** | **TP1_ELM** | 7.05% | -0.51% | -0.49% | -0.44% | -12.13% | -8.92% | -10.17% |
| | **TP2_ELM** | 9.93% | 2.60% | 2.62% | 2.67% | -8.66% | -5.55% | -6.76% |

### Full Results (Norm = Mean ± Std)

Out[7]:

| | BaseELM \| Train RMSE | BaseELM \| Test RMSE | BaseELM \| Norm | RidgeELM \| Train RMSE | RidgeELM \| Test RMSE | RidgeELM \| Norm | LiuELM \| Train RMSE | LiuELM \| Test RMSE | LiuELM \| Norm | OK_ELM \| Train RMSE | ... | TP2_ELM \| Norm | Ridge \| Train RMSE | Ridge \| Test RMSE | Ridge \| Norm | RandomForest \| Train RMSE | RandomForest \| Test RMSE | RandomForest \| Norm | XGBoost \| Train RMSE | XGBoost \| Test RMSE | XGBoost \| Norm |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Auto Price** | 1053.2533 ± 144.2096 | 25011.0056 ± 4697.3369 | 64428.7174 ± 8464.9479 | 1915.0152 ± 222.5482 | 11346.9151 ± 1253.0378 | 24372.2387 ± 2252.7006 | 1947.0071 ± 195.6867 | 11212.6067 ± 1225.2009 | 23752.3990 ± 1247.7040 | 1555.0300 ± 240.7876 | ... | 20670.7861 ± 1495.5639 | 2207.5822 ± 18.0156 | 2739.1416 ± 103.7826 | 3554.4250 ± 316.2556 | 1010.1970 ± 124.3703 | 2083.1541 ± 104.6628 | N/A | 485.4099 ± 318.2224 | 2073.5269 ± 173.5094 | N/A |
| **Boston Housing** | 9.6920 ± 0.7856 | 15.1660 ± 1.3113 | 39.0117 ± 2.4572 | 9.6986 ± 0.7855 | 14.9872 ± 1.2874 | 37.6511 ± 2.2323 | 9.6986 ± 0.7855 | 14.9872 ± 1.2874 | 37.6512 ± 2.2323 | 9.6984 ± 0.7855 | ... | 36.4356 ± 2.0425 | 4.5678 ± 0.0082 | 4.8325 ± 0.0495 | 7.4756 ± 0.3349 | 1.5677 ± 0.1337 | 3.3552 ± 0.1540 | N/A | 0.7345 ± 0.5124 | 3.1436 ± 0.2059 | N/A |
| **Fish** | 0.8334 ± 0.0403 | 1.2749 ± 0.1343 | 9.8780 ± 1.4495 | 0.8430 ± 0.0408 | 1.2253 ± 0.1205 | 7.7818 ± 0.9900 | 0.8433 ± 0.0407 | 1.2246 ± 0.1221 | 7.7675 ± 0.9855 | 0.8428 ± 0.0408 | ... | 7.0728 ± 0.9693 | 0.9447 ± 0.0003 | 0.9546 ± 0.0032 | 0.9137 ± 0.0031 | 0.4880 ± 0.0442 | 0.9027 ± 0.0104 | N/A | 0.5130 ± 0.1441 | 0.9105 ± 0.0142 | N/A |
| **Forests** | 55.9718 ± 0.9332 | 66.7399 ± 1.8228 | 55.5879 ± 2.7999 | 55.9727 ± 0.9332 | 66.4840 ± 1.8223 | 54.8308 ± 2.7524 | 55.9727 ± 0.9332 | 66.4840 ± 1.8223 | 54.8309 ± 2.7524 | 55.9727 ± 0.9332 | ... | 54.0992 ± 2.7083 | 60.8944 ± 0.6396 | 54.3633 ± 2.2408 | 38.1715 ± 10.4292 | 38.5426 ± 2.1850 | 58.7417 ± 2.9090 | N/A | 39.1354 ± 13.5247 | 58.9805 ± 8.6072 | N/A |
| **Machine CPU** | 3.1638 ± 0.0985 | 7998.9297 ± 5234.6181 | 5509903.3340 ± 1995669.4848 | 13.2663 ± 0.2605 | 19.9509 ± 1.7193 | 53.4335 ± 42.5382 | 13.3858 ± 0.1103 | 31.8446 ± 43.8325 | 6662.0237 ± 7798.2571 | 12.2892 ± 0.3176 | ... | 35.7833 ± 14.9377 | 15.5345 ± 0.0344 | 16.1675 ± 0.3114 | 3.8432 ± 0.3560 | 12.1753 ± 0.6451 | 17.2609 ± 0.5681 | N/A | 11.6916 ± 2.6471 | 17.0530 ± 1.1756 | N/A |
| **Servo** | 0.3568 ± 0.0463 | 2.3102 ± 0.5452 | 14.3979 ± 1.7777 | 0.5147 ± 0.0616 | 1.3852 ± 0.1656 | 4.6594 ± 0.5187 | 0.5150 ± 0.0537 | 1.3826 ± 0.1762 | 4.6349 ± 0.3693 | 0.5099 ± 0.0574 | ... | 3.9692 ± 0.6246 | 1.0248 ± 0.0026 | 1.1133 ± 0.0169 | 2.0061 ± 0.0581 | 0.5700 ± 0.0485 | 0.8240 ± 0.0512 | N/A | 0.2862 ± 0.1306 | 0.7114 ± 0.0895 | N/A |
| **Slump** | 0.0000 ± 0.0000 | 21.3950 ± 5.5802 | 66.3250 ± 11.1209 | 1.9436 ± 0.5597 | 15.1723 ± 2.9944 | 39.9735 ± 5.2716 | 2.0247 ± 0.5705 | 15.0721 ± 3.0081 | 39.5969 ± 5.1897 | 1.9185 ± 0.5519 | ... | 36.3342 ± 4.4940 | 2.3330 ± 0.0436 | 2.6246 ± 0.0864 | 9.3620 ± 0.4037 | 1.5505 ± 0.1296 | 3.6732 ± 0.1534 | N/A | 0.3097 ± 0.3787 | 3.3618 ± 0.2763 | N/A |
| **Strikes** | 432.9100 ± 5.6379 | 612.5706 ± 21.2774 | 3634.4354 ± 502.1274 | 438.4344 ± 5.4526 | 566.4720 ± 16.6945 | 1872.7070 ± 194.5223 | 438.4078 ± 5.4848 | 566.5731 ± 16.8101 | 1877.2757 ± 198.5402 | 438.2869 ± 5.4689 | ... | 1396.5119 ± 214.3760 | 524.0912 ± 1.1782 | 507.7516 ± 11.0515 | 199.6056 ± 1.1686 | 358.5270 ± 25.1955 | 522.7271 ± 13.5947 | N/A | 382.9539 ± 105.6924 | 516.8154 ± 16.8844 | N/A |

8 rows × 27 columns

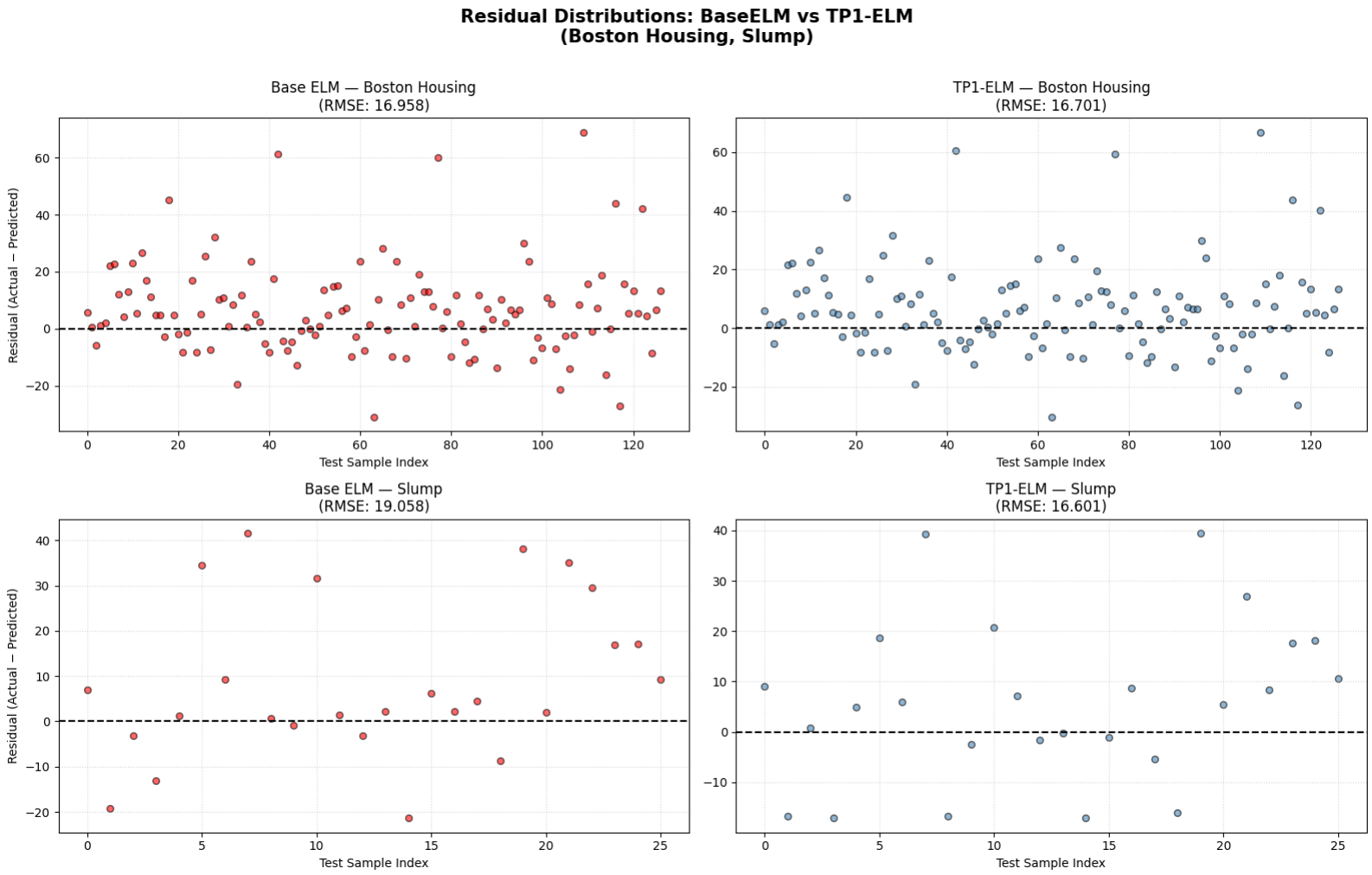
## Visualizing Residual Errors

To map performance bounds with mathematical rigidity, we compute isolated residual comparisons between `BaseELM` and `TP1_ELM` on both the **Boston Housing** and **Slump** datasets (mirroring Figures 7 and 8 in the paper). Tighter clustering of residuals around $y = 0$ confirms the TP1 regularization dampens the volatility caused by multicollinear interference in the base pseudo-inverse solution.

In [8]:

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
import numpy as np

def plot_residuals_for_dataset(ds_key, ax_base, ax_tp1, title_suffix=""):
    """Plot BaseELM vs TP1_ELM residuals for a given dataset key."""
    if ds_key not in datasets:
        ax_base.text(0.5, 0.5, f"{ds_key} not available", ha='center', transform=ax_base.transAxes)
        ax_tp1.text(0.5, 0.5, f"{ds_key} not available", ha='center', transform=ax_tp1.transAxes)
        return

    X_ds = datasets[ds_key]['X'].values
    y_ds = datasets[ds_key]['y']

    X_tr, X_te, y_tr, y_te = train_test_split(X_ds, y_ds, test_size=0.25, random_state=42)

    sc = StandardScaler()
    X_tr_s = sc.fit_transform(X_tr)
    X_te_s = sc.transform(X_te)

    # BaseELM
    base = BaseELM(n_hidden=100, random_state=42)
    base.fit(X_tr_s, y_tr)
    base_preds = base.predict(X_te_s)
    base_residuals = y_te.ravel() - base_preds.ravel()
    base_rmse = np.sqrt(mean_squared_error(y_te, base_preds))

    # TP1_ELM — brute-force mini grid search
    best_rmse_tp1 = float('inf')
    opt_tp1 = None
    for k in [1e-4, 1e-3, 0.05, 0.1, 0.5]:
        for d in [1e-4, 1e-3, 0.05, 0.1, 0.5]:
            temp = TP1_ELM(k=k, d=d, n_hidden=100, random_state=42)
            temp.fit(X_tr_s, y_tr)
            r = np.sqrt(mean_squared_error(y_te, temp.predict(X_te_s)))
            if r < best_rmse_tp1:
                best_rmse_tp1 = r
                opt_tp1 = temp
    tp1_preds = opt_tp1.predict(X_te_s)
```

```python
    tp1_residuals = y_te.ravel() - tp1_preds.ravel()

    idx = range(len(base_residuals))

    ax_base.scatter(idx, base_residuals, color='red', alpha=0.6, edgecolor='k', s=30)
    ax_base.axhline(0, color='black', linestyle='--', linewidth=1.5)
    ax_base.set_title(f"Base ELM — {ds_key}\n(RMSE: {base_rmse:.3f})", fontsize=12)
    ax_base.set_xlabel("Test Sample Index")
    ax_base.set_ylabel("Residual (Actual − Predicted)")
    ax_base.grid(True, linestyle=':', alpha=0.5)

    ax_tp1.scatter(idx, tp1_residuals, color='steelblue', alpha=0.6, edgecolor='k', s=30)
    ax_tp1.axhline(0, color='black', linestyle='--', linewidth=1.5)
    ax_tp1.set_title(f"TP1-ELM — {ds_key}\n(RMSE: {best_rmse_tp1:.3f})", fontsize=12)
    ax_tp1.set_xlabel("Test Sample Index")
    ax_tp1.grid(True, linestyle=':', alpha=0.5)


fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle(
    "Residual Distributions: BaseELM vs TP1-ELM\n"
    "(Boston Housing, Slump)",
    fontsize=15, fontweight='bold', y=1.01
)

# Boston Housing
plot_residuals_for_dataset('Boston Housing', axes[0, 0], axes[0, 1])

# Sump
plot_residuals_for_dataset('Slump', axes[1, 0], axes[1, 1])

plt.tight_layout()
plt.show()
```


Residual Distributions: BaseELM vs TP1-ELM
(Boston Housing, Slump)
Base ELM — Boston Housing
(RMSE: 16.958)
TP1-ELM — Boston Housing
(RMSE: 16.701)
Base ELM — Slump
(RMSE: 19.058)
TP1-ELM — Slump
(RMSE: 16.601)
Residual (Actual − Predicted)
Test Sample Index